

Neural Networks

EEE 443/543

Lecture 7-
Recurrent Neural Networks

Recurrent Neural Network Architectures

The fundamental feature of a *Recurrent Neural Network (RNN)* is that the network contains at least one *feed-back connection*, so the activations can flow round in a loop. That enables the networks to do *temporal processing* and *learn sequences*, e.g., perform sequence recognition/reproduction or temporal association/prediction.

Recurrent Neural Network Architectures

The fundamental feature of a *Recurrent Neural Network (RNN)* is that the network contains at least one *feed-back connection*, so the activations can flow round in a loop. That enables the networks to do *temporal processing* and *learn sequences*, e.g., perform sequence recognition/reproduction or temporal association/prediction.

Recurrent neural network architectures can have many different forms.

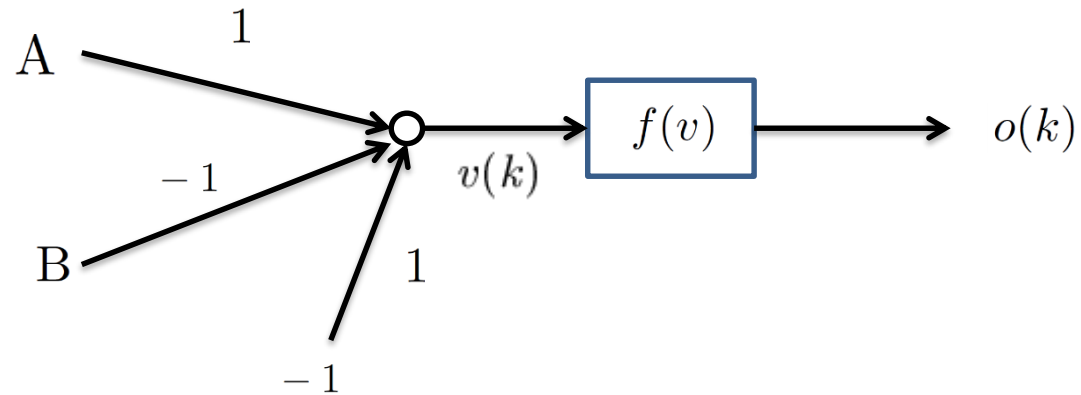
One common type consists of a standard Multi-Layer Perceptron (MLP) plus added loops. These can exploit the powerful non-linear mapping capabilities of the MLP, and also have some form of *memory*.

Others have more uniform structures, potentially with every neuron connected to all the others, and may also have stochastic activation functions.

<https://docplayer.net/20874007-Recurrent-neural-networks.html>

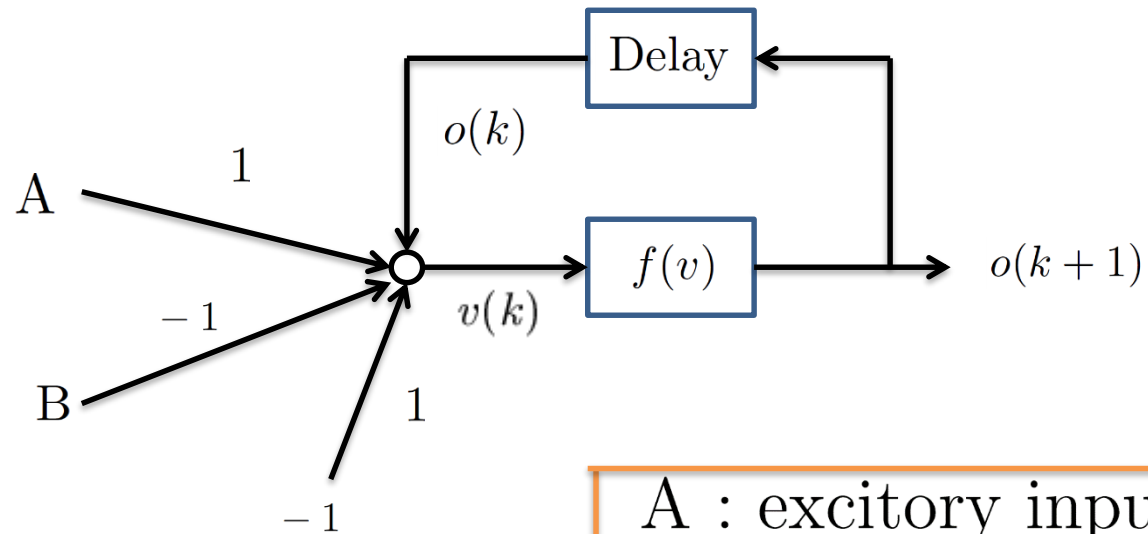
Feedback Networks :

Basic Memory Unit



Feedback Networks :

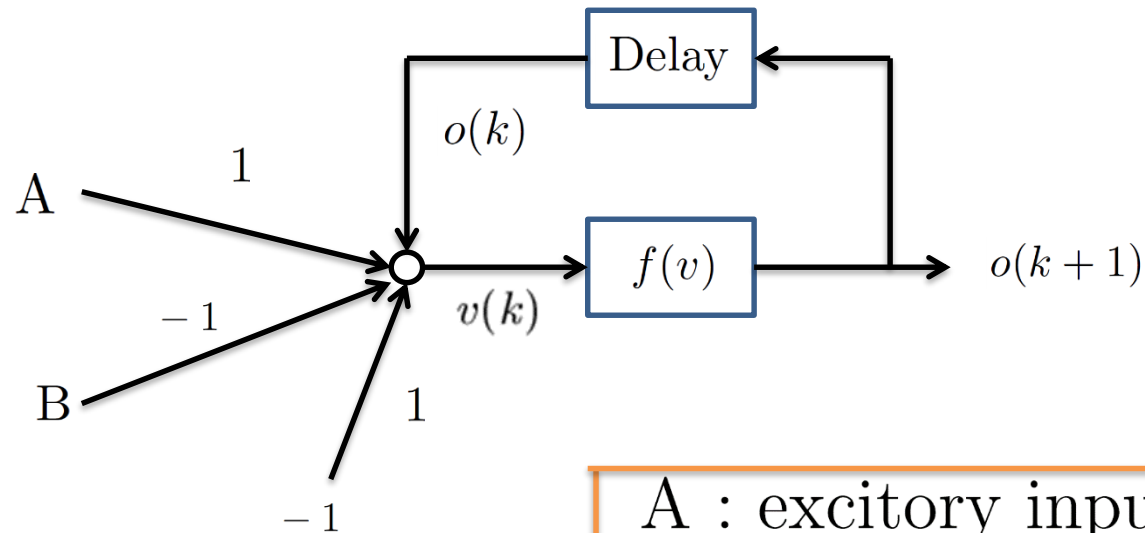
Basic Memory Unit



A : excitatory input (stores 1)
B : inhibitory input (stores 0)

Feedback Networks :

Basic Memory Unit

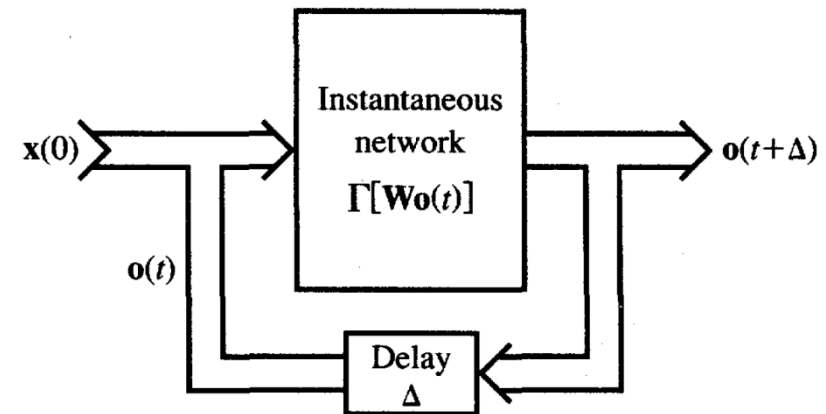
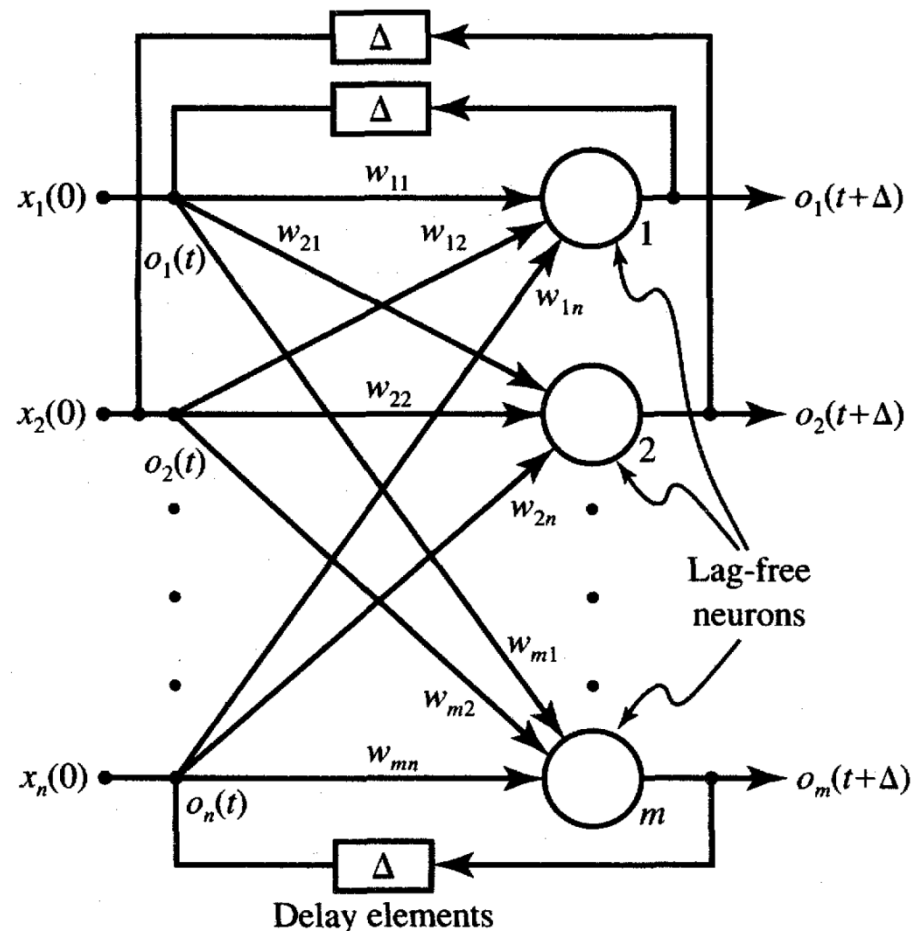


A : excitory input (stores 1)
B : inhibitory input (stores 0)

$$\left. \begin{aligned} v(k) &= o(k) + A - B - 1 \\ o(k+1) &= f(v(k)) \end{aligned} \right\}$$

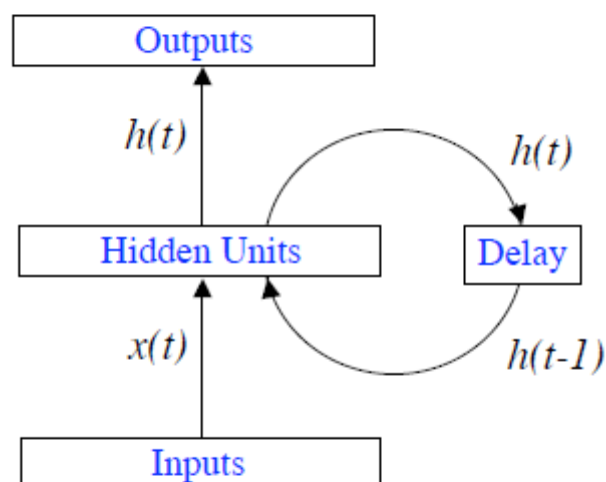
$$k = 0, 1, 2, \dots$$

A	B	$o(k)$	$v(k)$	$o(k + 1)$	Situation
0	0	0	-1	0	State is preserved
0	0	1	0	1	State is preserved
1	0	0	0	1	Excitation is preserved
1	0	1	1	1	Excitation is preserved
0	1	0	-2	0	Inhibition is preserved
0	1	1	-1	0	Inhibition is preserved
1	1	0	-1	0	State is preserved
1	1	1	0	1	State is preserved



A Fully Recurrent Network

The simplest form of *fully recurrent neural network* is an MLP with the previous set of hidden unit activations feeding back into the network along with the inputs:



Note that the time t has to be *discretized*, with the activations updated at each time step.

A *delay unit* needs to be introduced to hold activations until they are processed at the next time step.

The State Space Model

If the neural network inputs and outputs are the vectors $x(t)$ and $y(t)$, the three connection weight matrices are W_{IH} , W_{HH} and W_{HO} , and the hidden and output unit activation functions are f_H and f_O , the behaviour of the recurrent network can be described as a *dynamical system* by the pair of non-linear matrix equations:

$$h(t) = f_H(W_{IH}x(t) + W_{HH}h(t-1))$$

$$y(t) = f_O(W_{HO}h(t))$$

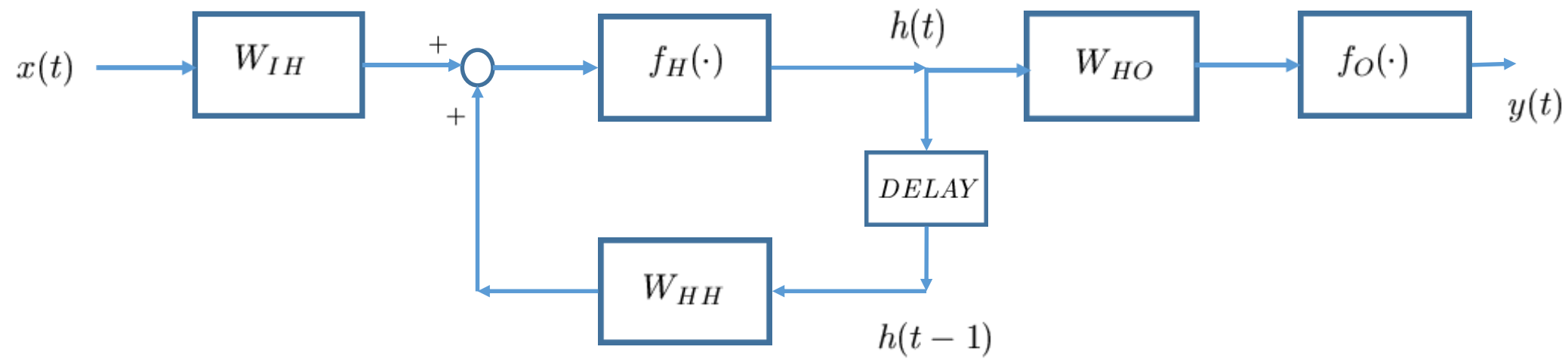
The State Space Model

If the neural network inputs and outputs are the vectors $x(t)$ and $y(t)$, the three connection weight matrices are W_{IH} , W_{HH} and W_{HO} , and the hidden and output unit activation functions are f_H and f_O , the behaviour of the recurrent network can be described as a *dynamical system* by the pair of non-linear matrix equations:

$$\begin{aligned}h(t) &= f_H(W_{IH}x(t) + W_{HH}h(t-1)) \\ y(t) &= f_O(W_{HO}h(t))\end{aligned}$$

The *state* of the dynamical system is defined by the set of hidden unit activations $h(t)$.

Thus, in addition to the input and output spaces, there is also a *state space*. The *order* of the dynamical system is the dimensionality of the state space, the number of hidden units.



Learning and Universal Approximation

The *Universal Approximation Theorem* tells us that:

Any non-linear dynamical system can be approximated to any accuracy by a recurrent neural network, with no restrictions on the compactness of the state space, provided that the network has enough sigmoidal hidden units.

This underlies the computational power of recurrent neural networks.

On The Computational Power Of Neural Nets

Hava T. Siegelmann
Department of Computer Science
Rutgers University
New Brunswick, NJ 08903
siegelma@paul.rutgers.edu

Eduardo D. Sontag
Department of Mathematics
Rutgers University
New Brunswick, NJ 08903
sontag@hilbert.rutgers.edu

Abstract

This paper deals with finite networks which consist of interconnections of synchronously evolving processors. Each processor updates its state by applying a “sigmoidal” scalar non-linearity to a linear combination of the previous states of all units. We prove that one may simulate all Turing Machines by rational nets. In particular, one can do this in linear time, and there is a net made up of about 1,000 processors which computes a universal partial-recursive function. Products (high order nets) are not required, contrary to what had been stated in the literature. Furthermore, we assert a similar theorem about non-deterministic Turing Machines. Consequences for undecidability and complexity issues about nets are discussed too.

but other nonlinearities are of course of interest as well. One of the processors is singled out as the “output node” of the net. Another processor, called the “validation node,” signals when the output occurs.

The use of sigmoidal functions —as opposed to hard thresholds— is what distinguishes this area from older work that dealt only with finite automata. Indeed, it has long been known, at least since the classical papers by McCulloch and Pitts ([12], [9]), how to implement logic gates by threshold networks, and therefore how to simulate finite automata by such nets. For us, however, nets are essentially *analog* computational devices, in accordance with models currently used in neural net practice.

In [14], Pollack argued that a certain recurrent net model, which he called a “neuring machine,” is universal. The model in [14] consisted of a finite number of neurons of two different kinds, having identity and threshold responses, respectively. The machine was

Minimal Width for Universal Property of Deep RNN

Chang hoon Song

*Department of Mathematical Science
Seoul National University*

GOLDBACH2@SNU.AC.KR

Geonho Hwang

*Department of Mathematical Science
Seoul National University*

HGH2134@SNU.AC.KR

Jun ho Lee

Kongju National University

JHLSAT@KONGJU.AC.KR

Myungjoo Kang

*Department of Mathematical Science
Seoul National University*

MKANG@SNU.AC.KR

Editor: Mehryar Mohri

[https://www.jmlr.org/
papers/volume24/22-
1191/22-1191.pdf](https://www.jmlr.org/papers/volume24/22-1191/22-1191.pdf)

Abstract

A recurrent neural network (RNN) is a widely used deep-learning network for dealing with sequential data. Imitating a dynamical system, an infinite-width RNN can approximate any open dynamical system in a compact domain. In general, deep narrow networks with bounded width and arbitrary depth are more effective than wide shallow networks with arbitrary width and bounded depth in practice; however, the universal approximation theorem for deep narrow structures has yet to be extensively studied. In this study, we prove the universality of deep narrow RNNs and show that the upper bound of the minimum width for universality can be independent of the length of the data. Specifically, we show a deep RNN with ReLU activation can approximate any continuous function or L^p function with the widths $d_x + d_y + 3$ and $\max\{d_x + 1, d_y\}$, respectively, where the target function maps a finite sequence of vectors in $\mathbb{R}^{d_x}$ to a finite sequence of vectors in $\mathbb{R}^{d_y}$. We also compute the additional width required if the activation function is sigmoid or more. In addition, we prove the universality of other recurrent networks, such as bidirectional RNNs. Bridging a multi-layer perceptron and an RNN, our theory and technique can shed light on further research on deep RNNs.

Keywords: recurrent neural network, universal approximation, deep narrow network, sequential data, minimum width

Stability, Controllability and Observability

Since one can think about recurrent networks in terms of their properties as dynamical systems, it is natural to ask about their *stability*, *controllability* and *observability*:

Stability concerns the boundedness over time of the network outputs, and the response of the network outputs to small changes (e.g., to the network inputs or weights).

Controllability is concerned with whether it is possible to control the dynamic behaviour. A recurrent neural network is said to be *controllable* if an initial state is steerable to any desired state within a finite number of time steps.

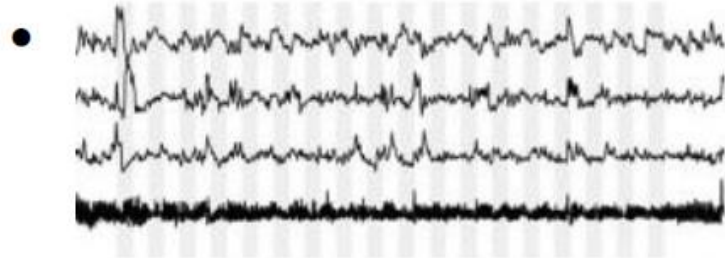
Observability is concerned with whether it is possible to observe the results of the control applied. A recurrent network is said to be *observable* if the state of the network can be determined from a finite set of input/output measurements.

A rigorous treatment of these issues is way beyond the scope of this module!

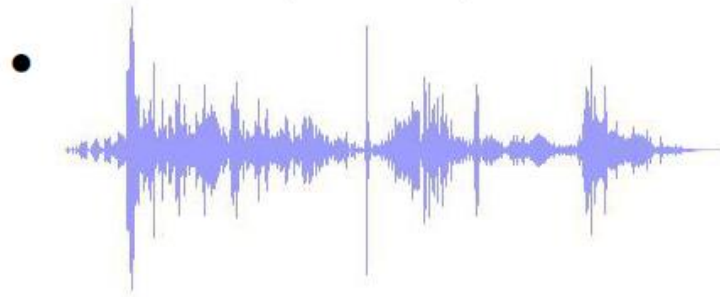
Sequential data

- “I took the dog for a walk this morning.”

sentence



medical signals

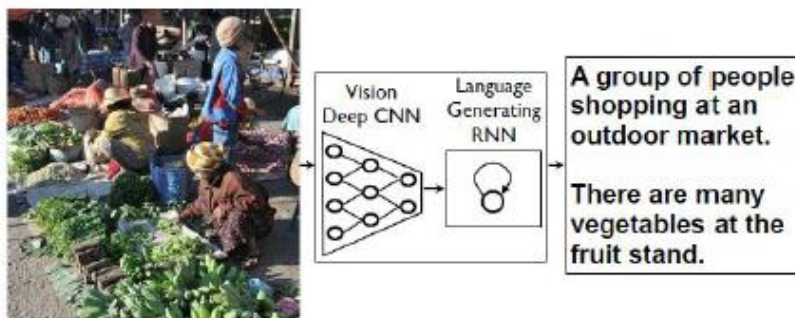


speech waveform



video frames

- Sample data sequences from a certain distribution $P(x_1, \dots, x_N)$
- Generate natural sentences to describe an image $P(y_1, \dots, y_M | I)$



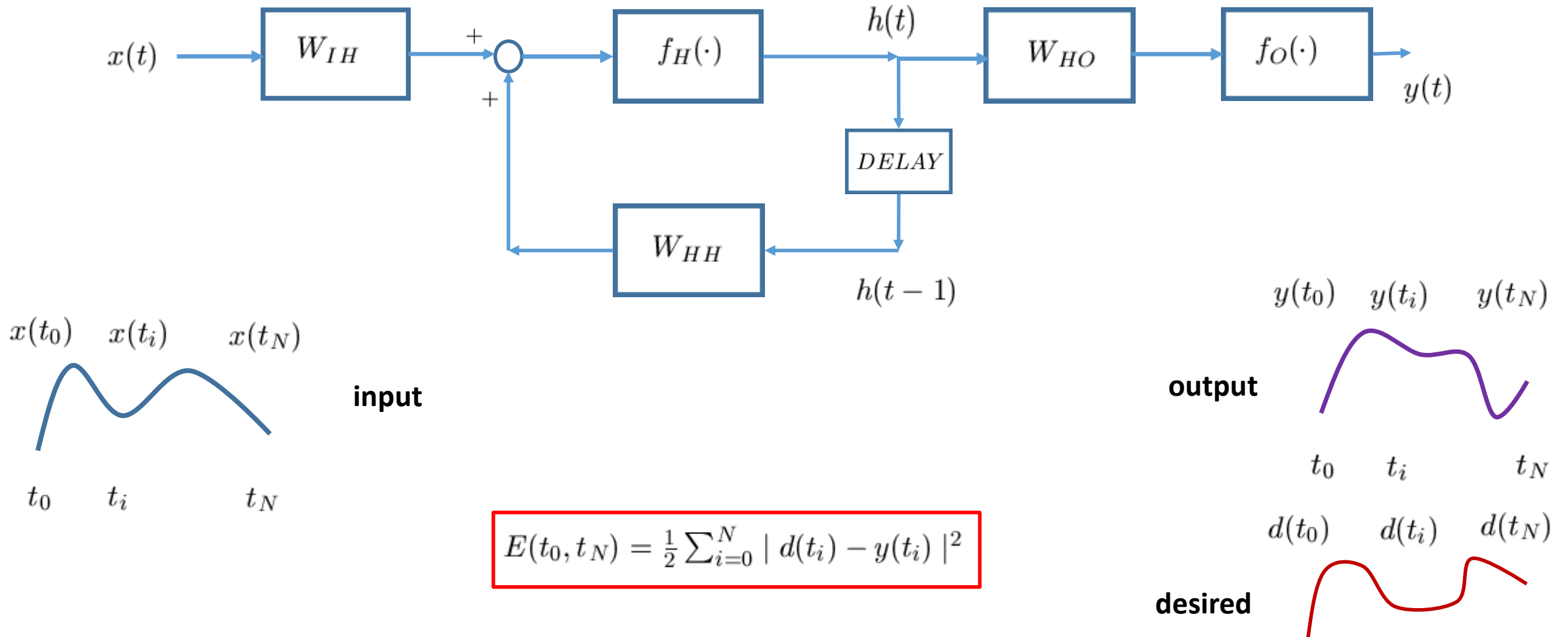
- Activity recognition from a video sequence



$$P(y | x_1, \dots, x_N)$$

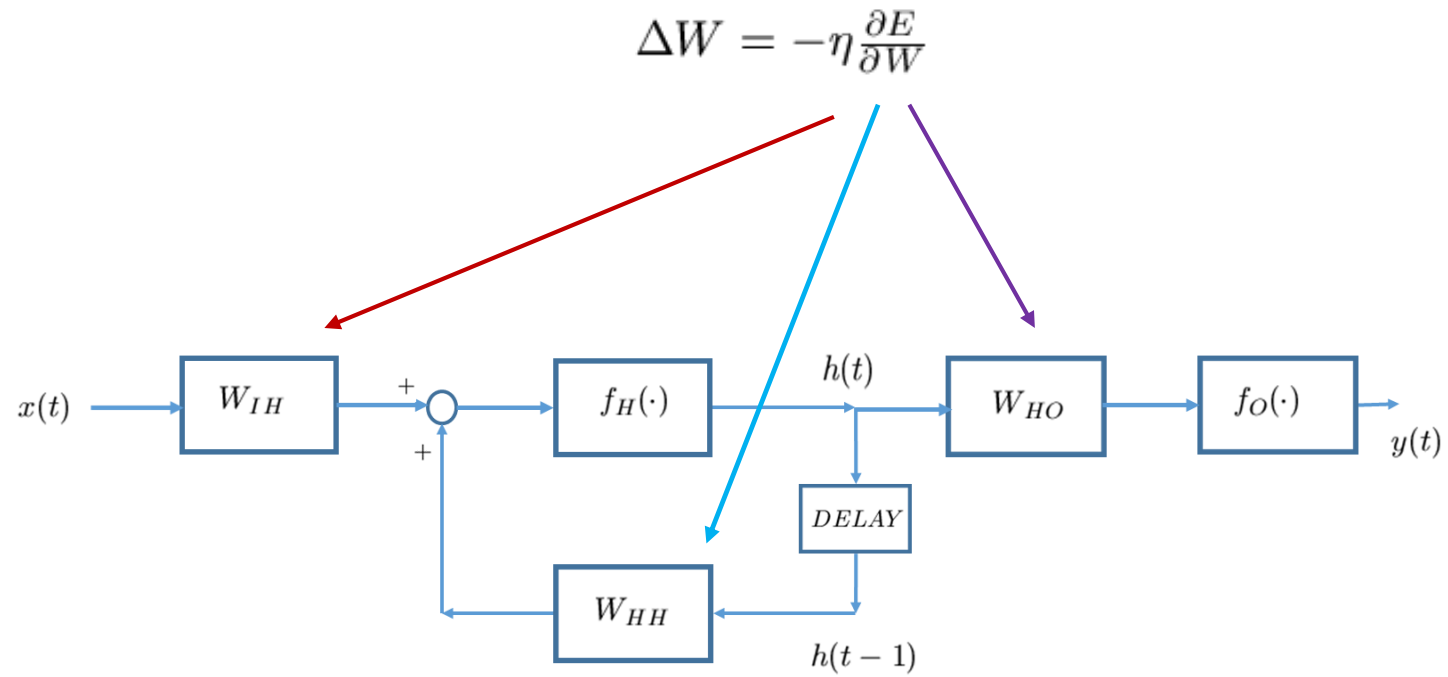
- ☐ Running
- ☒ Jumping
- ☐ Dancing
- ☐ Fighting
- ☐ Eating

How to train ?



How to apply back propagation ?

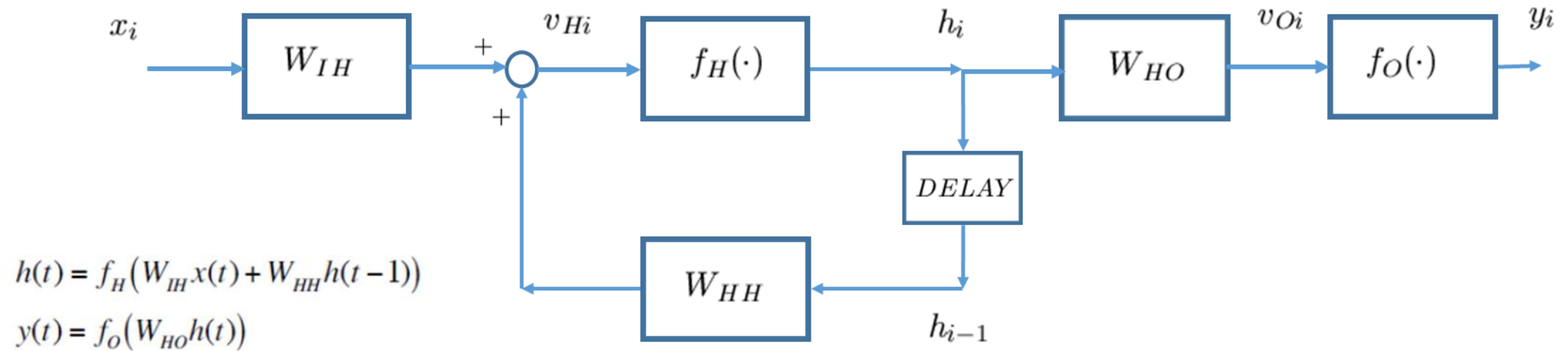
$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$



$$W \leftarrow W - \eta \frac{\partial E}{\partial W}$$

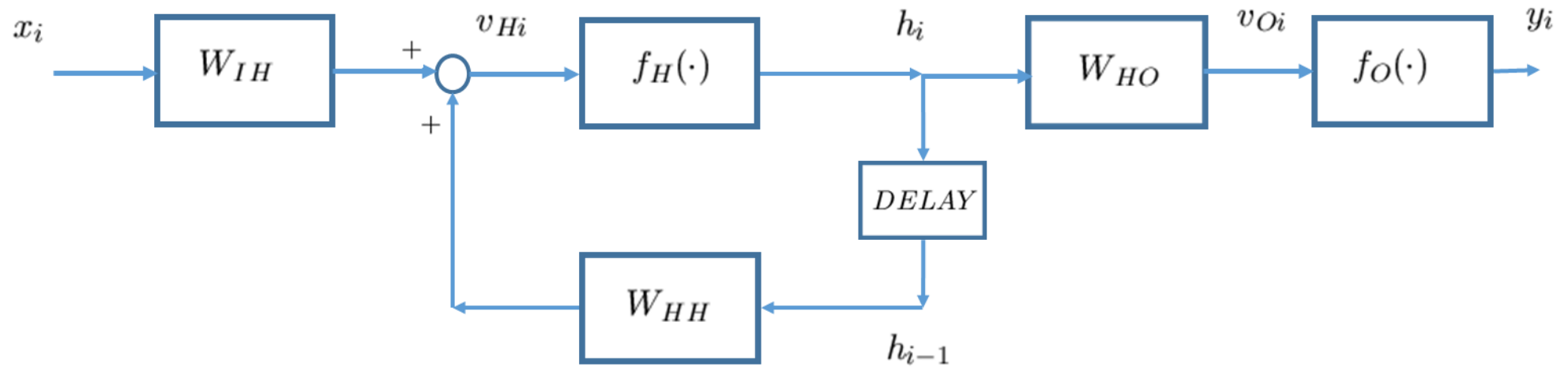
Forward Propagation

$$z_i = z(t_i) \quad i = 0, 1, \dots, N$$



Back Propagation

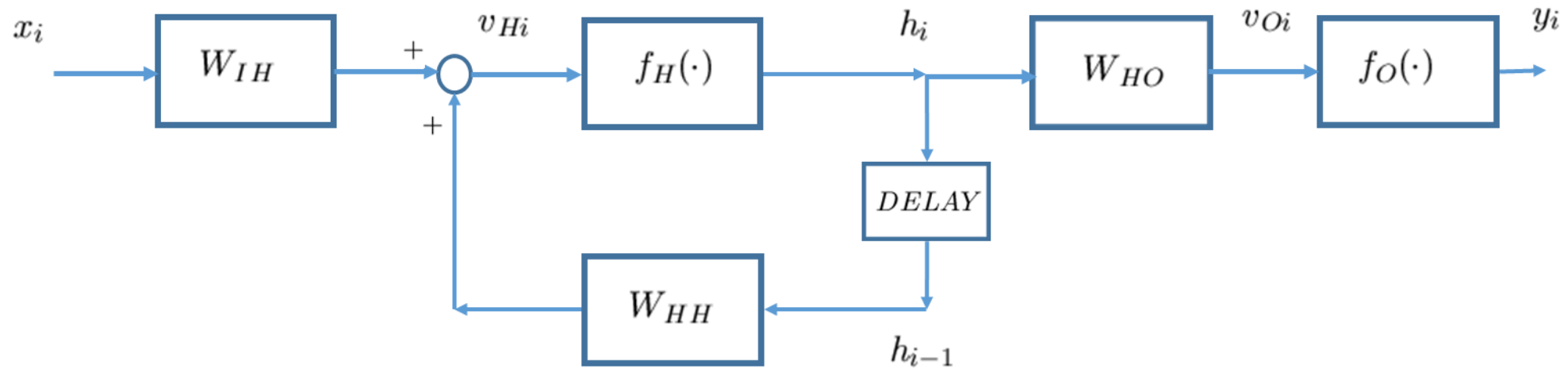
$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$



Back Propagation

$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$

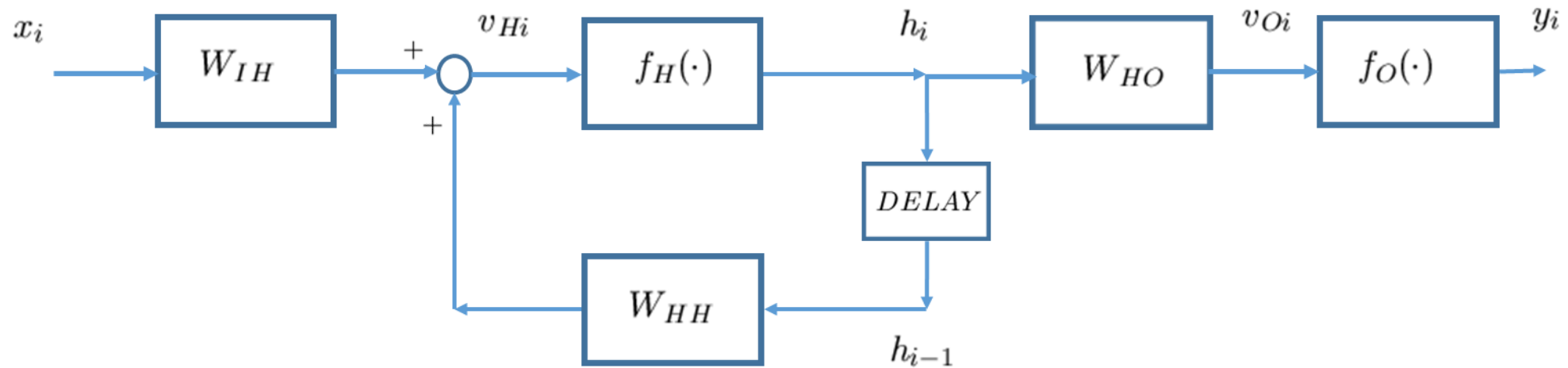
$$E \rightarrow e_i \rightarrow y_i \rightarrow v_{Oi} \rightarrow W_{HO}$$



Back Propagation

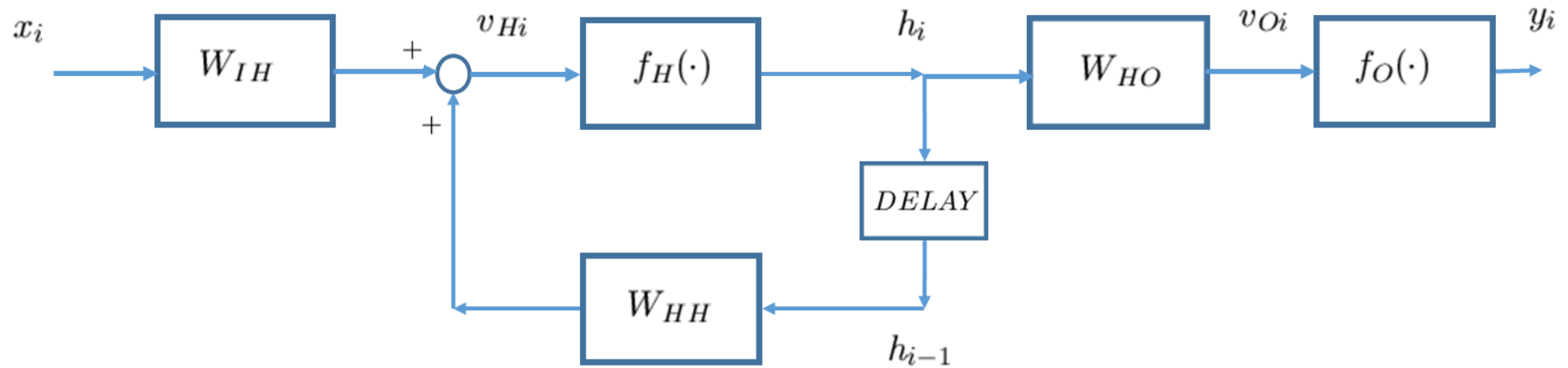
$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$

$$E \rightarrow e_i \rightarrow y_i \rightarrow v_{Oi} \rightarrow W_{HO} \quad \longrightarrow \quad \frac{\partial E}{\partial e_i} \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial v_{Oi}} \frac{\partial v_{Oi}}{\partial W_{HO}}$$



Back Propagation

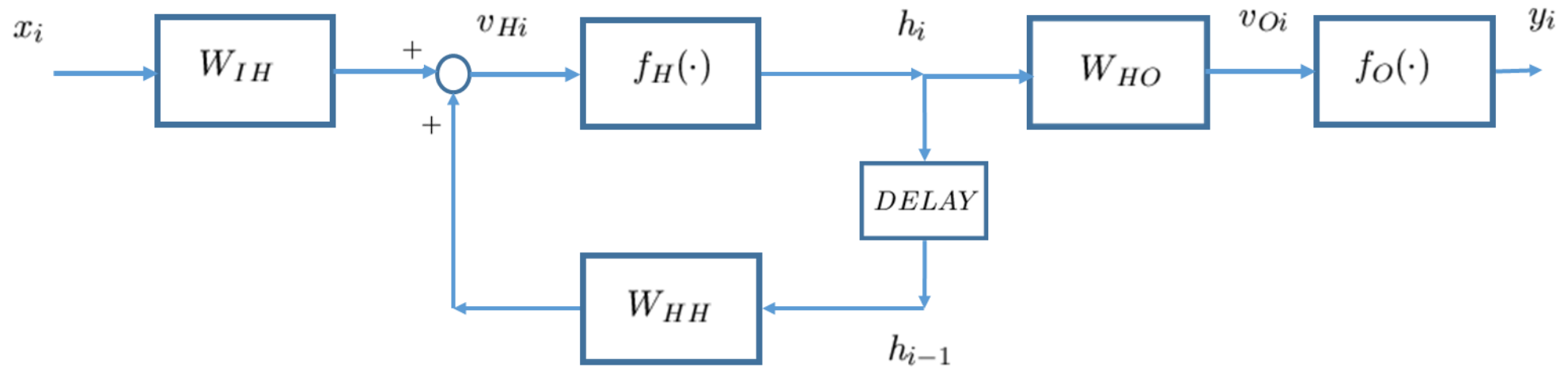
$$\begin{aligned}\frac{\partial E}{\partial W_{HO}} &= \sum_{i=0}^N \frac{\partial E}{\partial e_i} \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial v_{Oi}} \frac{\partial v_{Oi}}{\partial W_{HO}} \\ &= \sum_{i=0}^N e_i(-1)f'_{HO}(v_{Oi})h_i\end{aligned}$$



Back Propagation

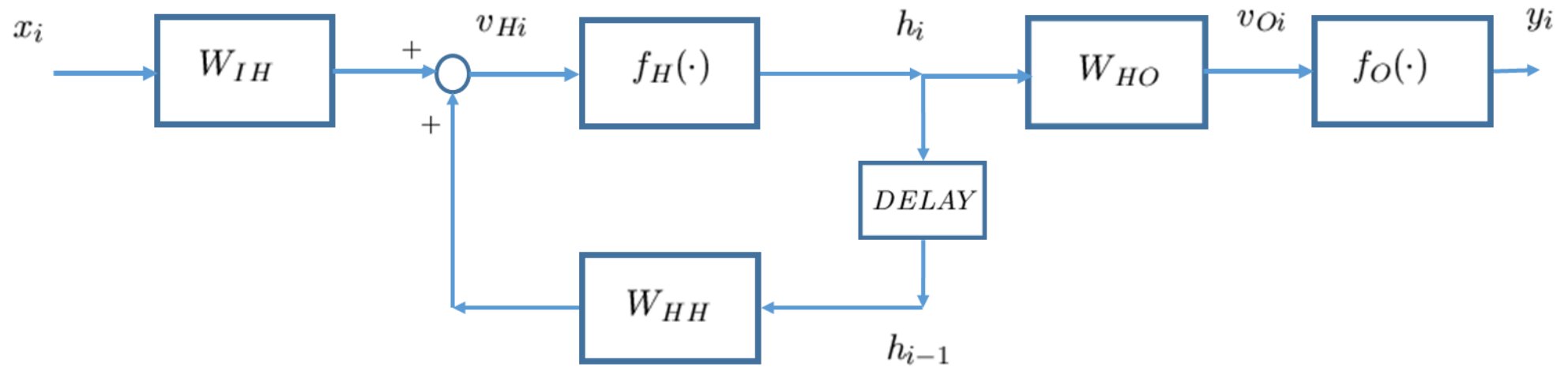
Be careful on the indices when $W_{HO} \in \mathbf{R}^{m \times R}$

$$\begin{aligned}\frac{\partial E}{\partial W_{HO}} &= \sum_{i=0}^N \frac{\partial E}{\partial e_i} \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial v_{Oi}} \frac{\partial v_{Oi}}{\partial W_{HO}} \\ &= \sum_{i=0}^N e_i(-1) f'_{HO}(v_{Oi}) h_i\end{aligned}$$



Back Propagation

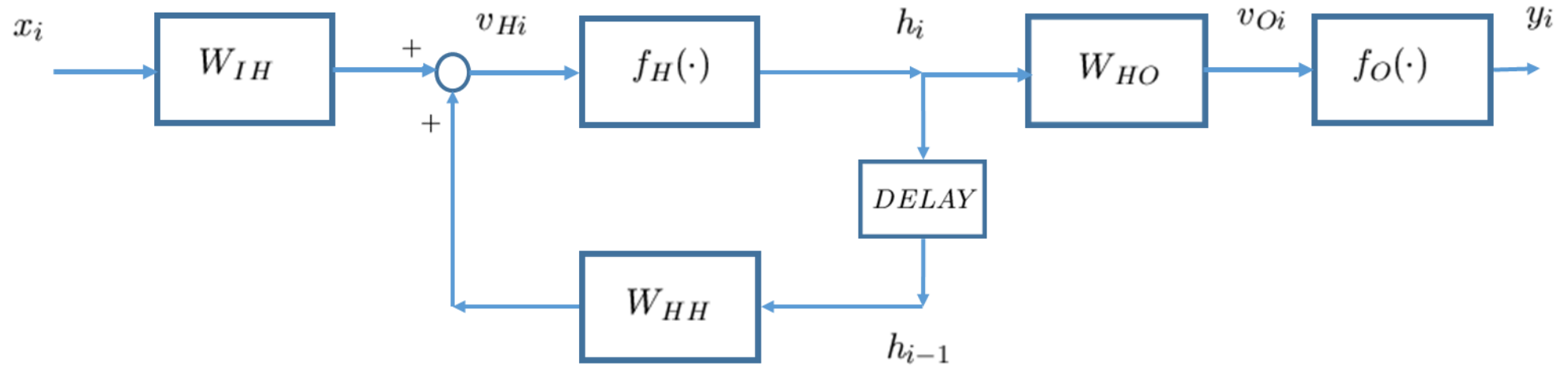
$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$



Back Propagation

$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$

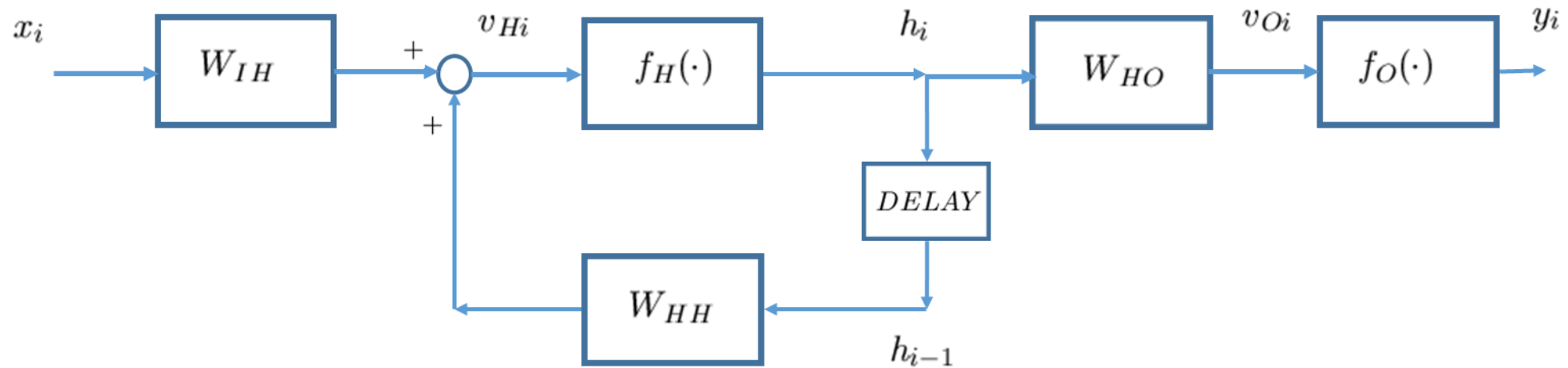
$$E \rightarrow e_i \rightarrow y_i \rightarrow v_{Oi} \rightarrow h_i \rightarrow v_{Hi} \rightarrow W_{HH}$$



Back Propagation

$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$

$$E \rightarrow e_i \rightarrow y_i \rightarrow v_{Oi} \rightarrow h_i \rightarrow v_{Hi} \rightarrow W_{HH} \quad \Rightarrow \quad \frac{\partial e_i}{\partial W_{HH}} = \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial v_{Oi}} \frac{\partial v_{Oi}}{\partial h_i} \frac{\partial h_i}{\partial v_{Hi}} \frac{\partial v_{Hi}}{\partial W_{HH}}$$

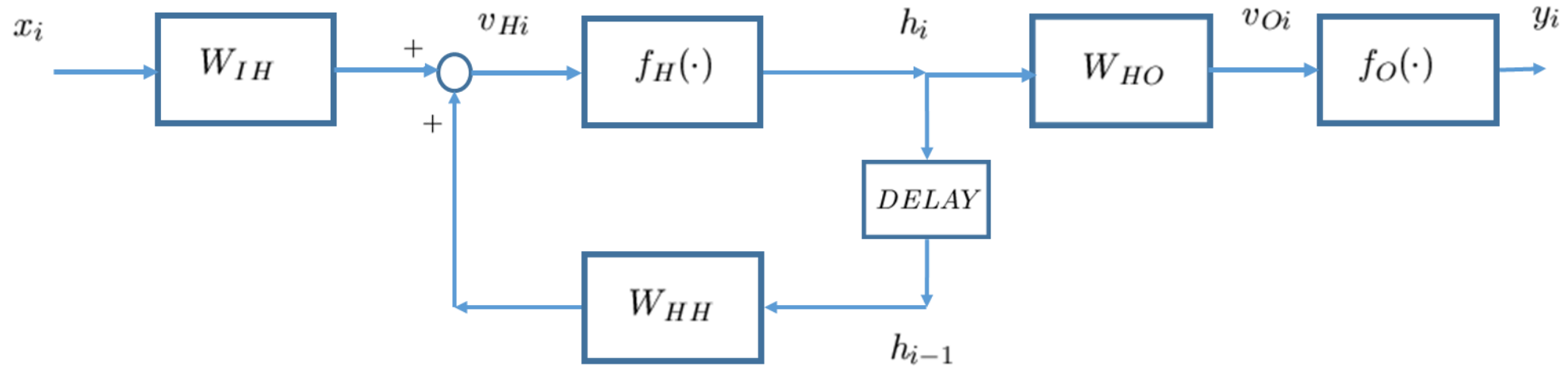


Back Propagation

$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$

$$E \rightarrow e_i \rightarrow y_i \rightarrow v_{Oi} \rightarrow h_i \rightarrow v_{Hi} \rightarrow W_{HH} \quad \Rightarrow \quad \frac{\partial e_i}{\partial W_{HH}} = \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial v_{Oi}} \frac{\partial v_{Oi}}{\partial h_i} \frac{\partial h_i}{\partial v_{Hi}} \frac{\partial v_{Hi}}{\partial W_{HH}}$$

$$h_i = f_H(W_{HI}x_i + W_{HH}h_{i-1})$$



Back Propagation

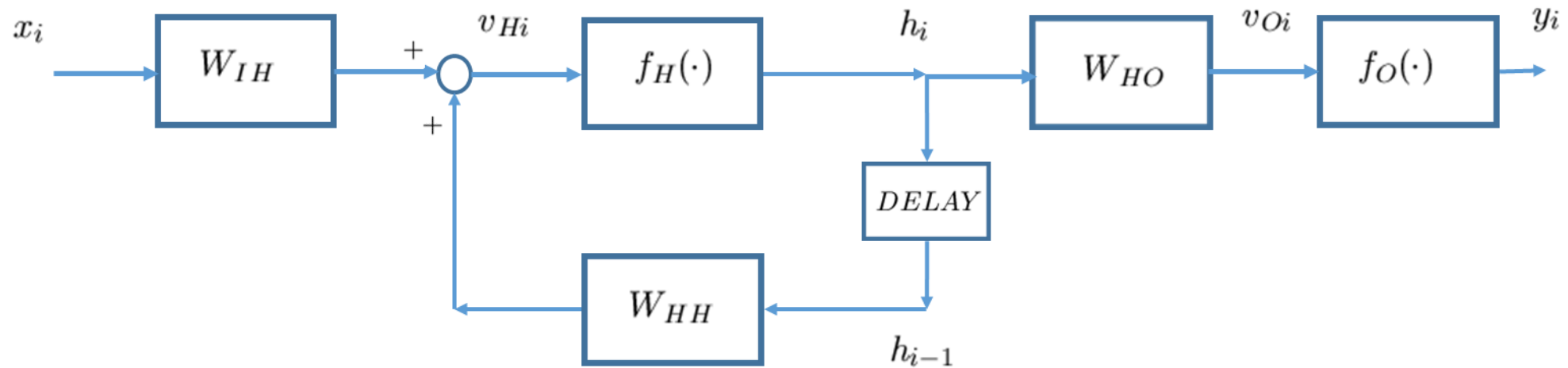
$$e_i = d(t_i) - y(t_i) \quad , \quad E = E(t_0, t_N) = \frac{1}{2} \sum_{i=0}^N e_i^2$$

$$E \rightarrow e_i \rightarrow y_i \rightarrow v_{Oi} \rightarrow h_i \rightarrow v_{Hi} \rightarrow W_{HH} \quad \Rightarrow \quad \frac{\partial e_i}{\partial W_{HH}} = \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial v_{Oi}} \frac{\partial v_{Oi}}{\partial h_i} \frac{\partial h_i}{\partial v_{Hi}} \frac{\partial v_{Hi}}{\partial W_{HH}}$$

$$h_i = f_H(W_{HI}x_i + W_{HH}h_{i-1})$$

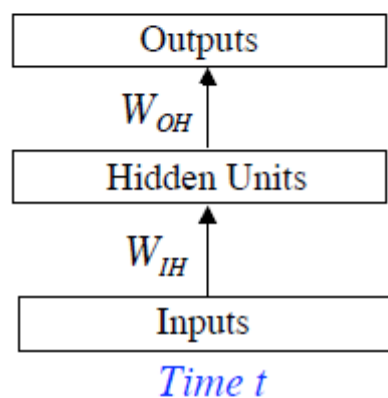
$$h_{i-1} = f_H(W_{HI}x_{i-1} + W_{HH}h_{i-2})$$

$\vdots$



Unfolding Over Time

The recurrent network can be converted into a feed-forward network by *unfolding over time*:

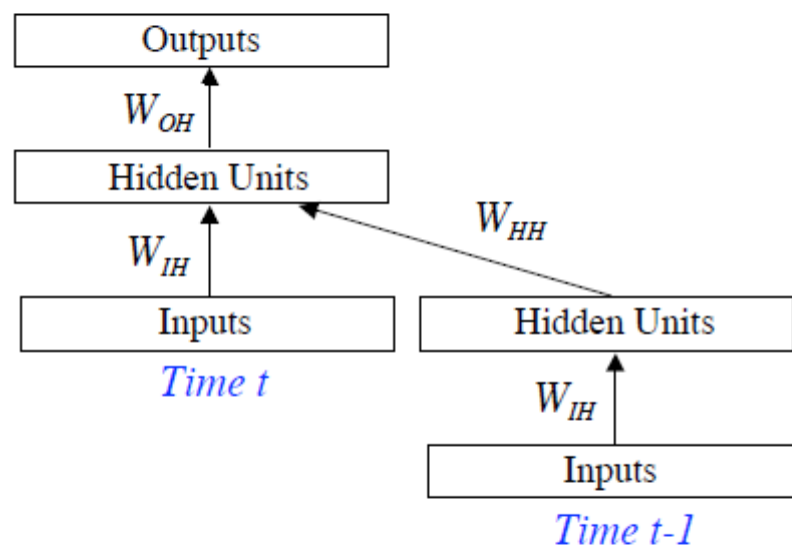


$$h(t) = f_H(W_{IH}x(t) + W_{HH}h(t-1))$$

$$y(t) = f_O(W_{HO}h(t))$$

Unfolding Over Time

The recurrent network can be converted into a feed-forward network by *unfolding over time*:

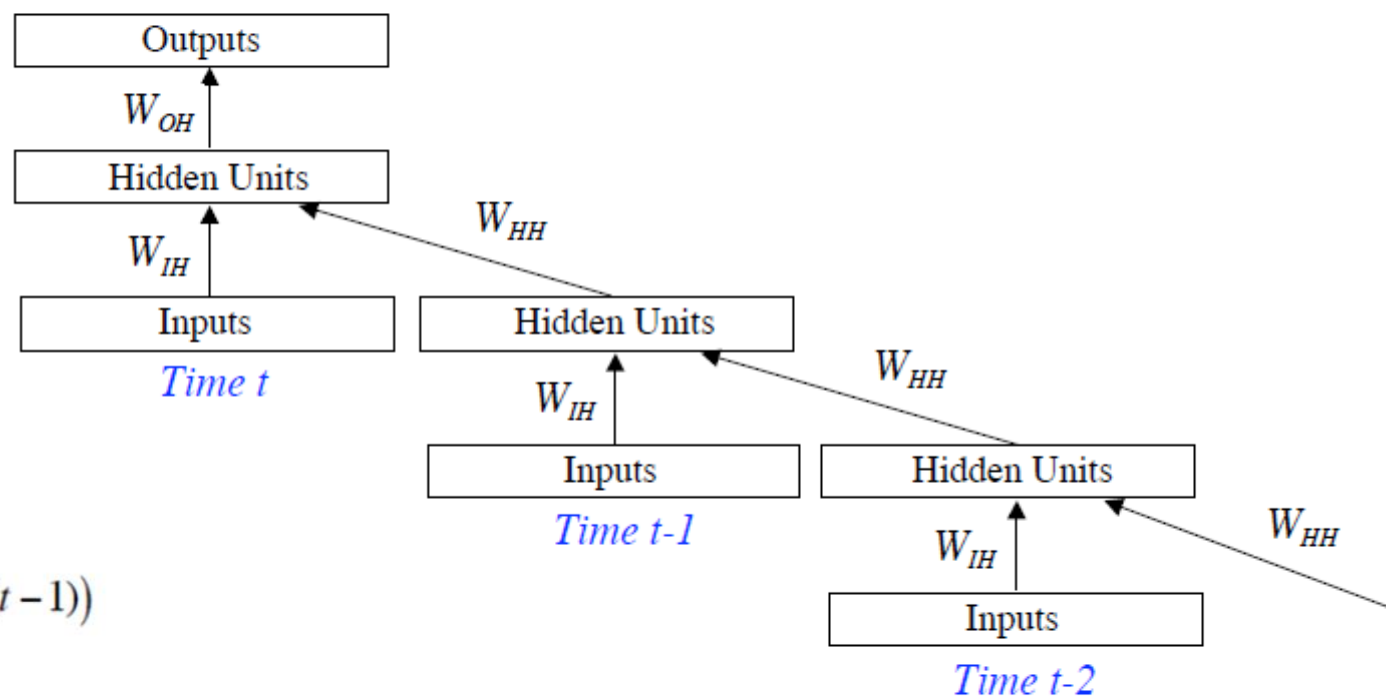


$$h(t) = f_H(W_{IH}x(t) + W_{HH}h(t-1))$$

$$y(t) = f_O(W_{HO}h(t))$$

Unfolding Over Time

The recurrent network can be converted into a feed-forward network by *unfolding over time*:



$$h(t) = f_H(W_{IH}x(t) + W_{HH}h(t-1))$$

$$y(t) = f_O(W_{HO}h(t))$$

This means all the earlier theory about feed-forward network learning follows through.

Backpropagation Through Time

The Backpropagation Through Time (*BPTT*) learning algorithm is a natural extension of standard backpropagation that performs gradient descent on a complete unfolded network.

Backpropagation Through Time

The Backpropagation Through Time (*BPTT*) learning algorithm is a natural extension of standard backpropagation that performs gradient descent on a complete unfolded network.

If a network training sequence starts at time t_0 and ends at time t_1 , the total cost function is simply the sum over time of the standard error function $E_{sse/ce}(t)$ at each time-step:

$$E_{total}(t_0, t_1) = \sum_{t=t_0}^{t_1} E_{sse/ce}(t)$$

Backpropagation Through Time

The Backpropagation Through Time (*BPTT*) learning algorithm is a natural extension of standard backpropagation that performs gradient descent on a complete unfolded network.

If a network training sequence starts at time t_0 and ends at time t_1 , the total cost function is simply the sum over time of the standard error function $E_{sse/ce}(t)$ at each time-step:

$$E_{total}(t_0, t_1) = \sum_{t=t_0}^{t_1} E_{sse/ce}(t)$$

and the gradient descent weight updates have contributions from each time-step

$$\Delta w_{ij} = -\eta \frac{\partial E_{total}(t_0, t_1)}{\partial w_{ij}} = -\eta \sum_{t=t_0}^{t_1} \frac{\partial E_{sse/ce}(t)}{\partial w_{ij}}$$

Backpropagation Through Time

The Backpropagation Through Time (*BPTT*) learning algorithm is a natural extension of standard backpropagation that performs gradient descent on a complete unfolded network.

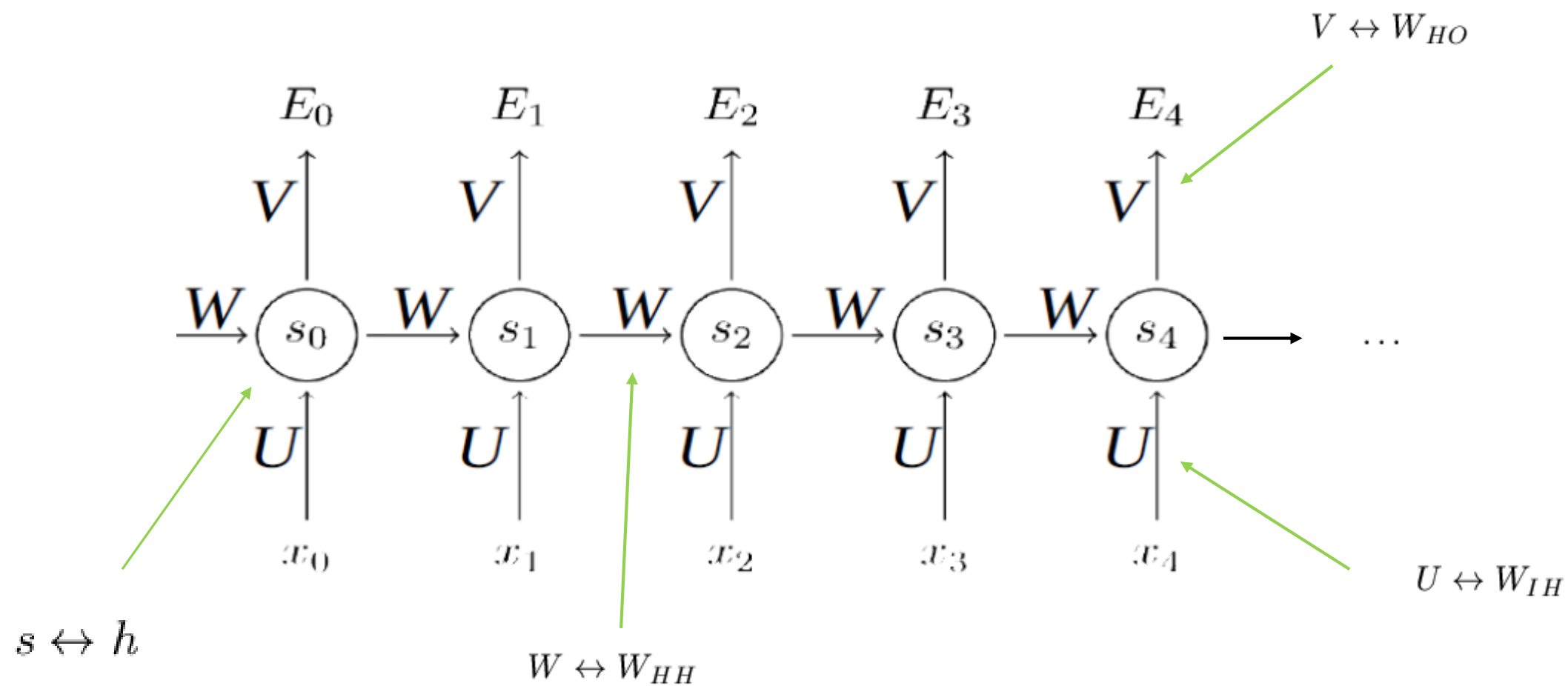
If a network training sequence starts at time t_0 and ends at time t_1 , the total cost function is simply the sum over time of the standard error function $E_{sse/ce}(t)$ at each time-step:

$$E_{total}(t_0, t_1) = \sum_{t=t_0}^{t_1} E_{sse/ce}(t)$$

and the gradient descent weight updates have contributions from each time-step

$$\Delta w_{ij} = -\eta \frac{\partial E_{total}(t_0, t_1)}{\partial w_{ij}} = -\eta \sum_{t=t_0}^{t_1} \frac{\partial E_{sse/ce}(t)}{\partial w_{ij}}$$

The constituent partial derivatives $\partial E_{sse/ce} / \partial w_{ij}$ now have contributions from the multiple instances of each weight $w_{ij} \in \{W_{IH}, W_{HH}\}$ and depend on the inputs and hidden unit activations at previous time steps. The errors now have to be back-propagated through time as well as through the network.



Backpropagation Through Time

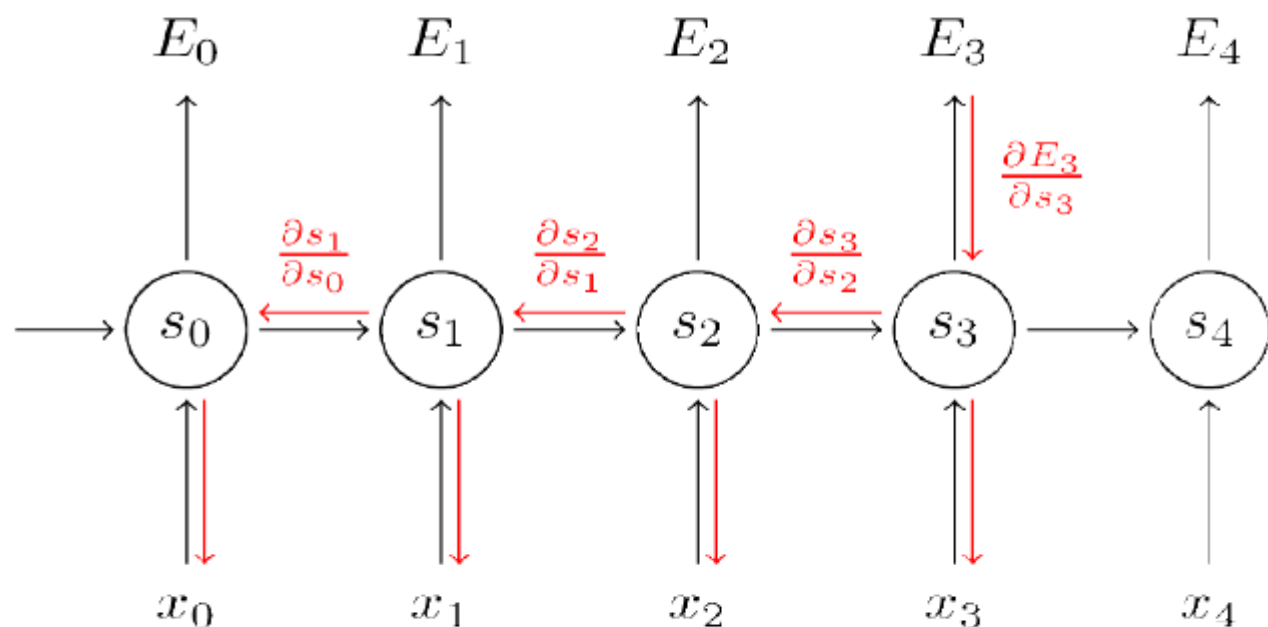
$$\frac{\partial E}{\partial \mathbf{W}} = \sum_t \frac{\partial E_t}{\partial \mathbf{W}}$$

$$\frac{\partial E_3}{\partial \mathbf{W}} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial \mathbf{W}}$$

But $s_3 = \tanh(Ux_t + Ws_2)$

s_3 depends on s_2 , which depends on W and s_1 , and so on.

$$\frac{\partial E_3}{\partial \mathbf{W}} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial \mathbf{W}}$$



$$\frac{\partial E_3}{\partial \mathbf{W}} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial \mathbf{W}}$$

$$\frac{\partial E_3}{\partial \mathbf{W}} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \left(\prod_{j=k+1}^3 \frac{\partial s_j}{\partial s_{j-1}} \right) \frac{\partial s_k}{\partial \mathbf{W}}$$

$$\frac{\partial h_{t+1}}{\partial h_k} = \frac{\partial h_{t+1}}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

$$\frac{\partial h_{t+1}}{\partial h_k} = \frac{\partial h_{t+1}}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

$$\frac{\partial E}{\partial W_{IH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}}$$

https://iq.opengenus.org/back-propagation-through-time/#google_vignette

<https://mmuratarat.github.io/2019-02-07/bptt-of-rnn>

Generalization of Back-Propagation to Recurrent Neural Networks

Fernando J. Pineda

Applied Physics Laboratory, Johns Hopkins University, Laurel, Maryland 20707

(Received 10 June 1987)

An adaptive neural network with asymmetric connections is introduced. This network is related to the Hopfield network with graded neurons and uses a recurrent generalization of the δ rule of Rumelhart, Hinton, and Williams to modify adaptively the synaptic weights. The new network bears a resemblance to the master/slave network of Lapedes and Farber, but it is architecturally simpler.

PACS numbers: 87.30.Gy

The neural network approach is a paradigm for computation in which the traditional paradigm of a finite-state machine performing sequential instructions in a discrete state space is replaced with the paradigm of a dynamical system, in a discrete or continuous state space, which evolves under the control of a certain class of dynamics (*neurodynamics*). Although a precise definition of neurodynamics does not exist, it seems safe to characterize it by at least three salient features. First, the dynamical system has very many degrees of freedom

tem parameters. This general approach is reviewed by Amari² and forms the basis of many learning algorithms. The algorithm described here is a specific case of this general approach.

The dynamics of the network considered in this Letter is based on the following system of coupled differential equations:

$$dx_i/dt = -\alpha x_i + \beta f_i \left(\sum_j w_{ij} x_j \right) + I_i, \quad (1)$$

Backpropagation Through Time: What It Does and How to Do It

PAUL J. WERBOS

Backpropagation is now the most widely used tool in the field of artificial neural networks. At the core of backpropagation is a method for calculating derivatives exactly and efficiently in any large system made up of elementary subsystems or calculations which are represented by known, differentiable functions; thus, backpropagation has many applications which do not involve neural networks as such.

This paper first reviews basic backpropagation, a simple method which is now being widely used in areas like pattern recognition and fault diagnosis. Next, it presents the basic equations for backpropagation through time, and discusses applications to areas like pattern recognition involving dynamic systems, systems identification, and control. Finally, it describes further extensions of this method, to deal with systems other than neural networks, systems involving simultaneous equations or true recurrent networks, and other practical issues which arise with this method. Pseudocode is provided to clarify the algorithms. The chain rule for ordered derivatives—the theorem which underlies backpropagation—is briefly discussed.

propagation.” The concepts here will already be familiar to those who have read the paper by Rumelhart, Hinton, and Williams [2] in the seminal book *Parallel Distributed Processing*, which played a pivotal role in the development of the field. (That book also acknowledged the prior work of Parker [3] and Le Cun [4], and the pivotal role of Charles Smith of the Systems Development Foundation.) This section will use new notation which adds a bit of generality and makes it easier to go on to complex applications in a rigorous manner. (The need for new notation may seem unnecessary to some, but for those who have to *apply* backpropagation to complex systems, it is essential.)

Section III will use the same notation to describe backpropagation through time. Backpropagation through time has been applied to concrete problems by a number of

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=58337>

On the difficulty of training Recurrent Neural Networks

Razvan Pascanu

Universite de Montreal

Tomas Mikolov

Brno University

Yoshua Bengio

Universite de Montreal

PASCANUR@IRO.UMONTREAL.CA

T.MIKOLOV@GMAIL.COM

YOSHUA.BENGIO@UMONTREAL.CA

Abstract

There are two widely known issues with properly training Recurrent Neural Networks, the *vanishing* and the *exploding* gradient problems detailed in Bengio *et al.* (1994). In this paper we attempt to improve the understanding of the underlying issues by exploring these problems from an analytical, a geometric and a dynamical systems perspective. Our analysis is used to justify a simple yet effective solution. We propose a gradient norm clipping strategy to deal with exploding gradients and a soft constraint for the vanishing gradients problem. We validate empirically our hypothesis and proposed solutions in the experimental section.

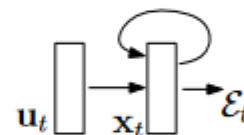


Figure 1. Schematic of a recurrent neural network. The recurrent connections in the hidden layer allow information to persist from one input to another.

and *exploding gradient* problems described in Bengio *et al.* (1994).

1.1. Training recurrent networks

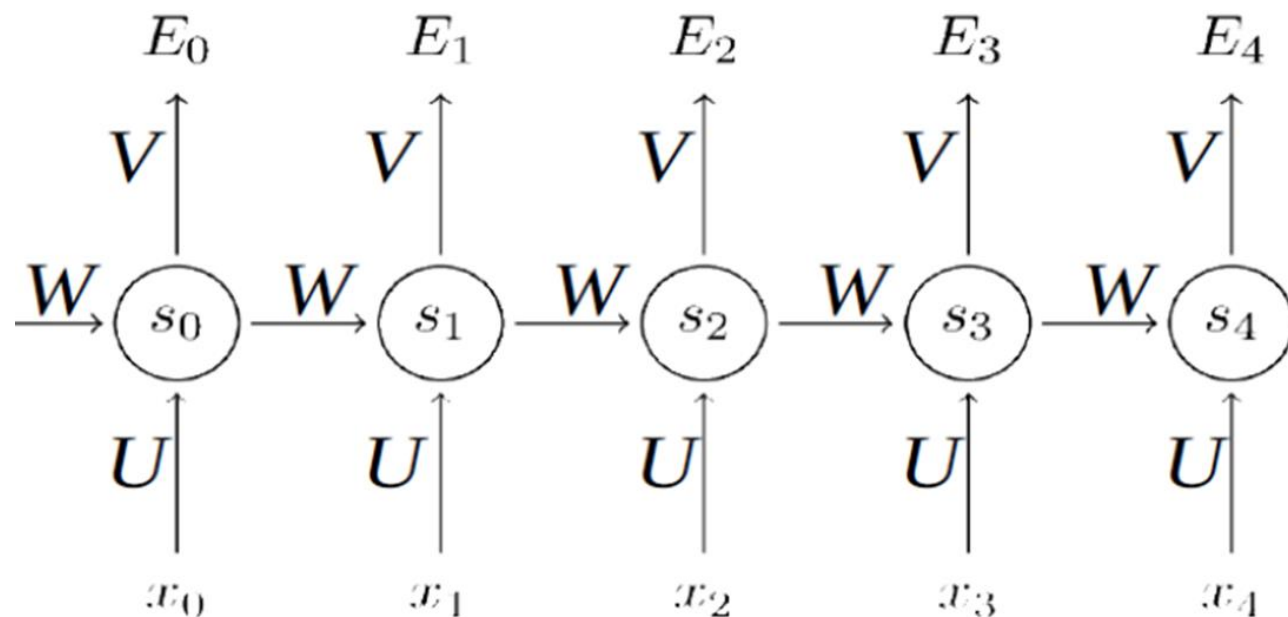
A generic recurrent neural network, with input u_t and state x_t for time step t , is given by equation (1). In the theoretical section of this paper we will sometimes

$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}} \end{aligned}$$

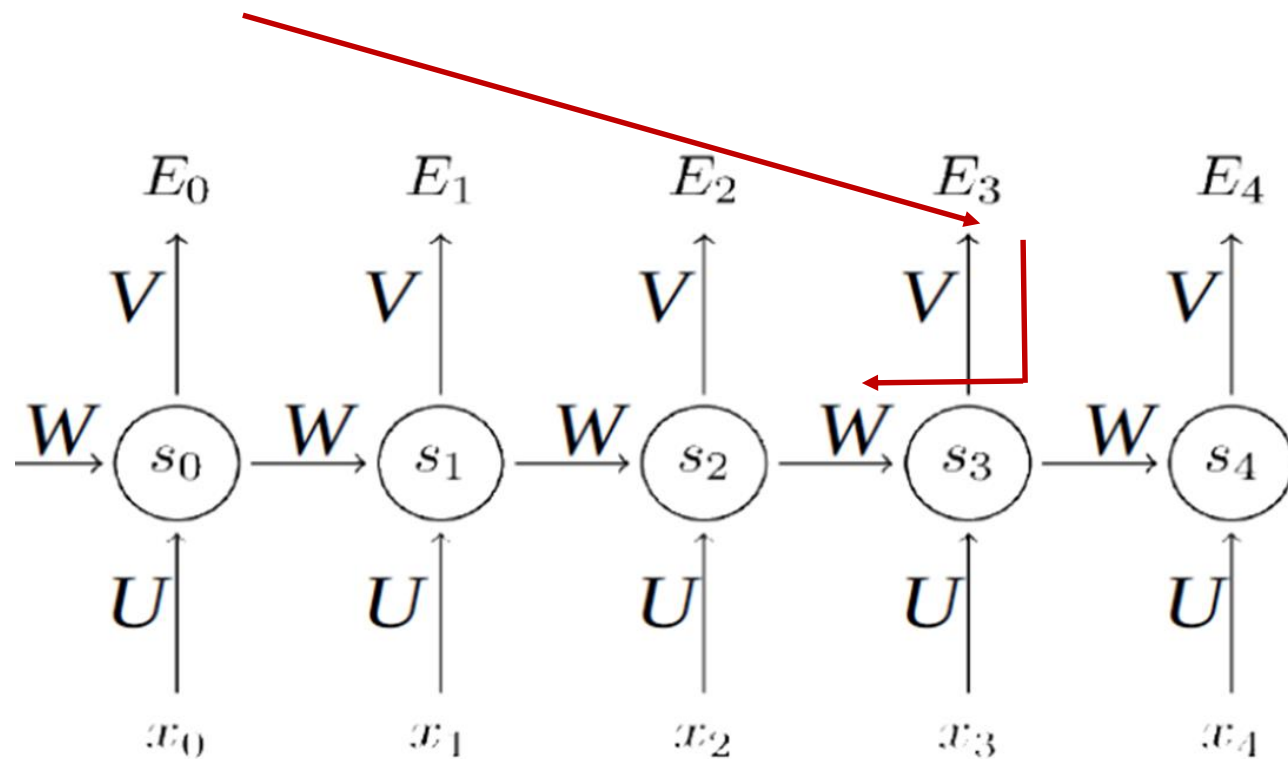
$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}} \end{aligned}$$



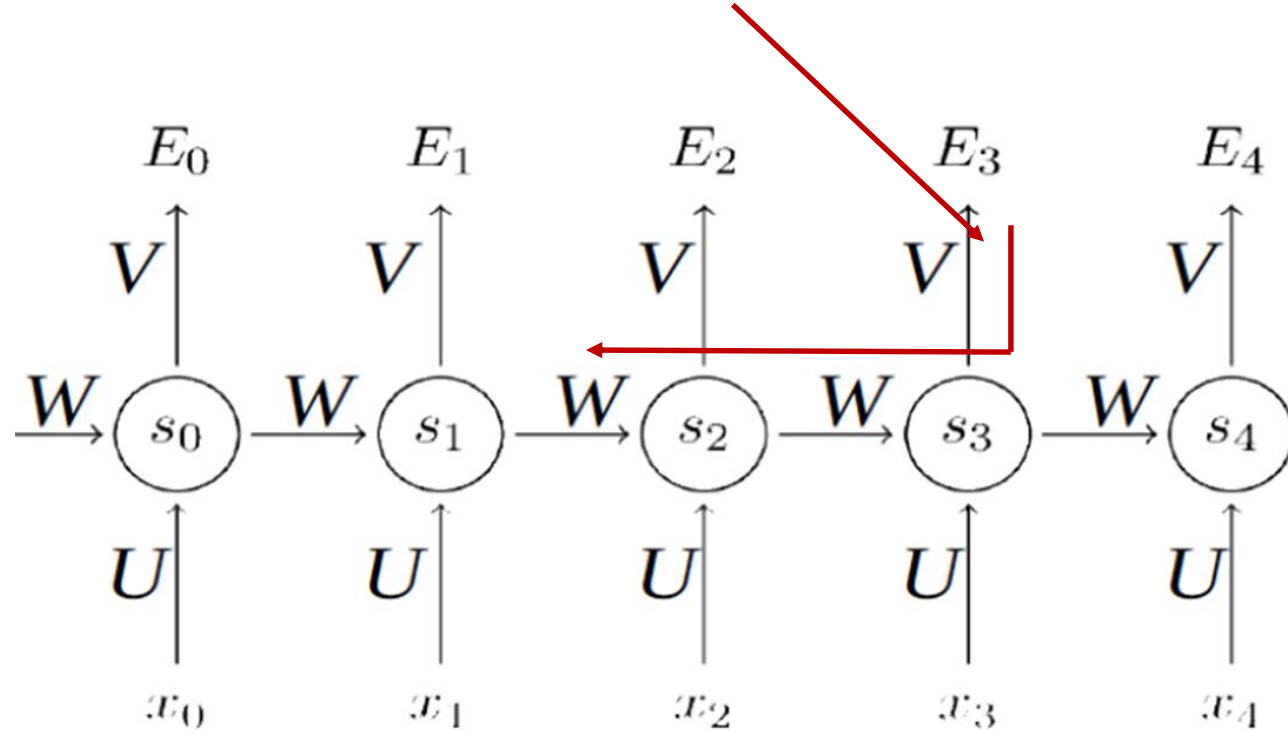
$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}} \end{aligned}$$



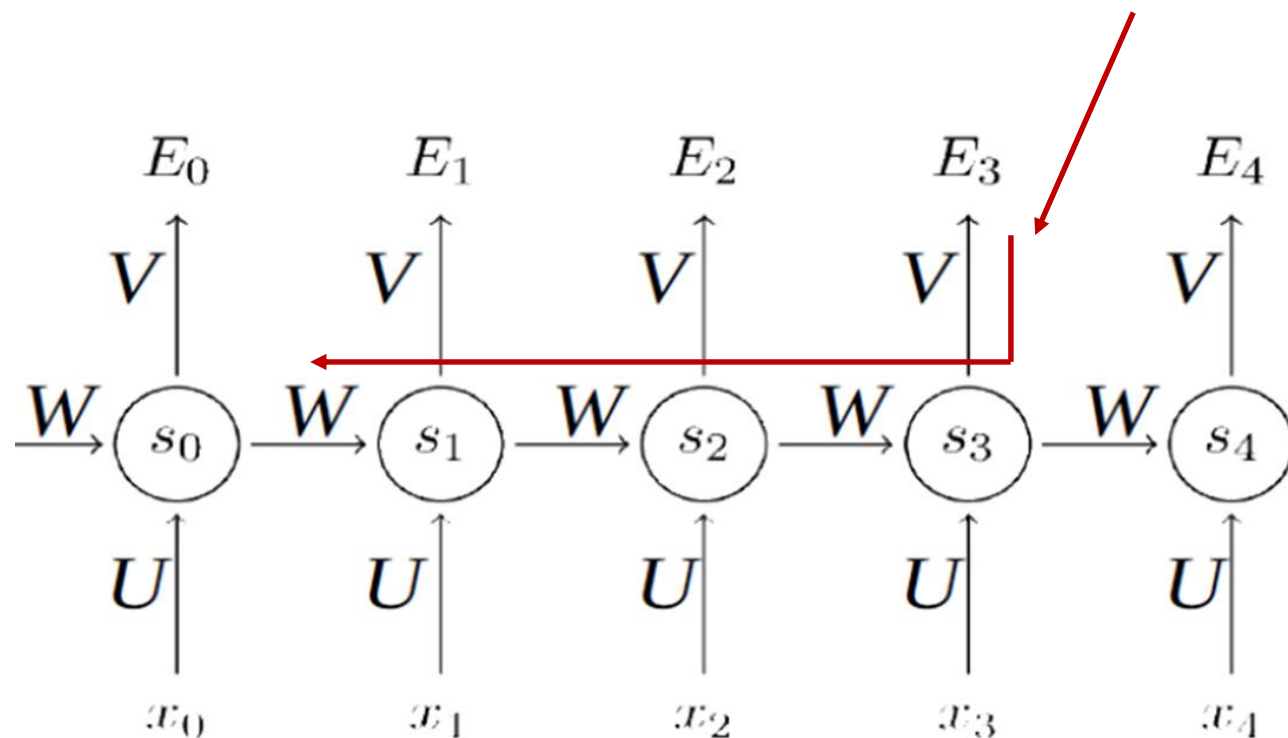
$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}} \end{aligned}$$



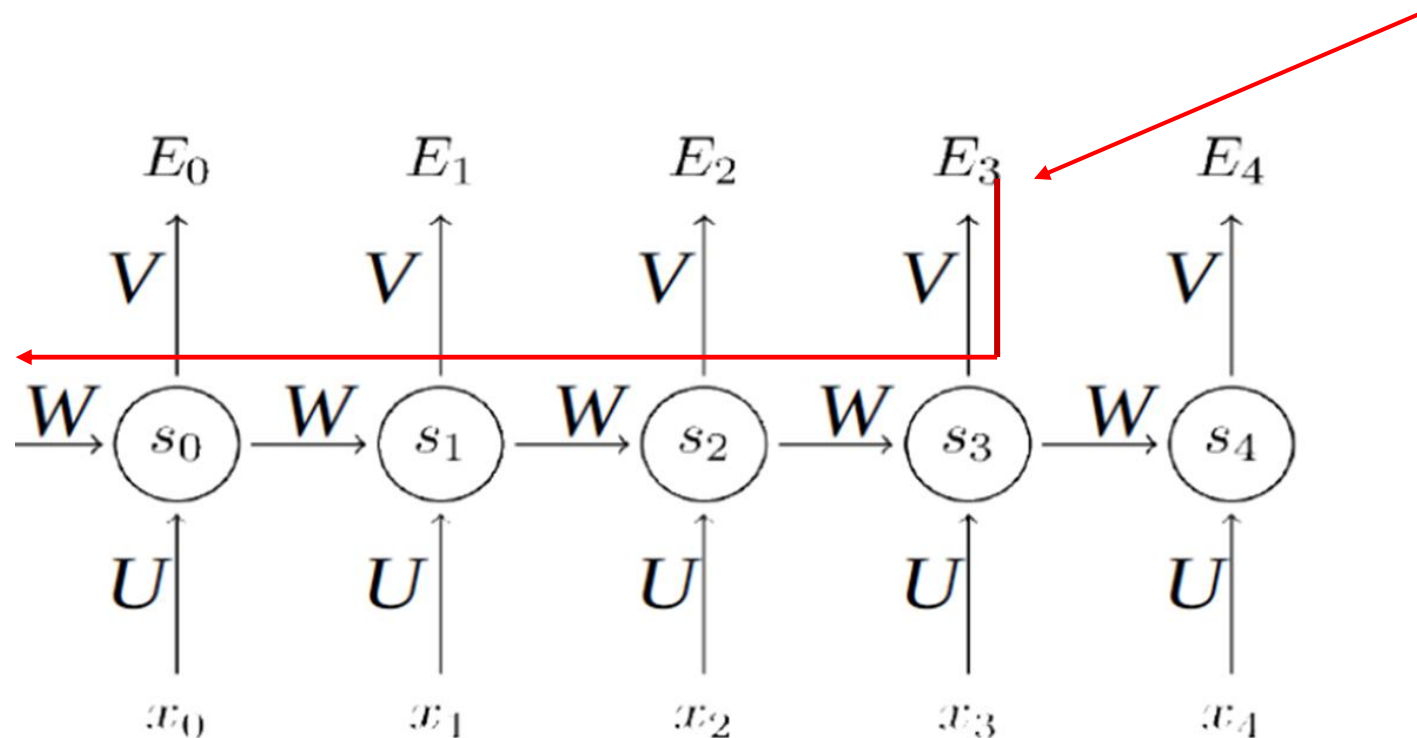
$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}} \end{aligned}$$



$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

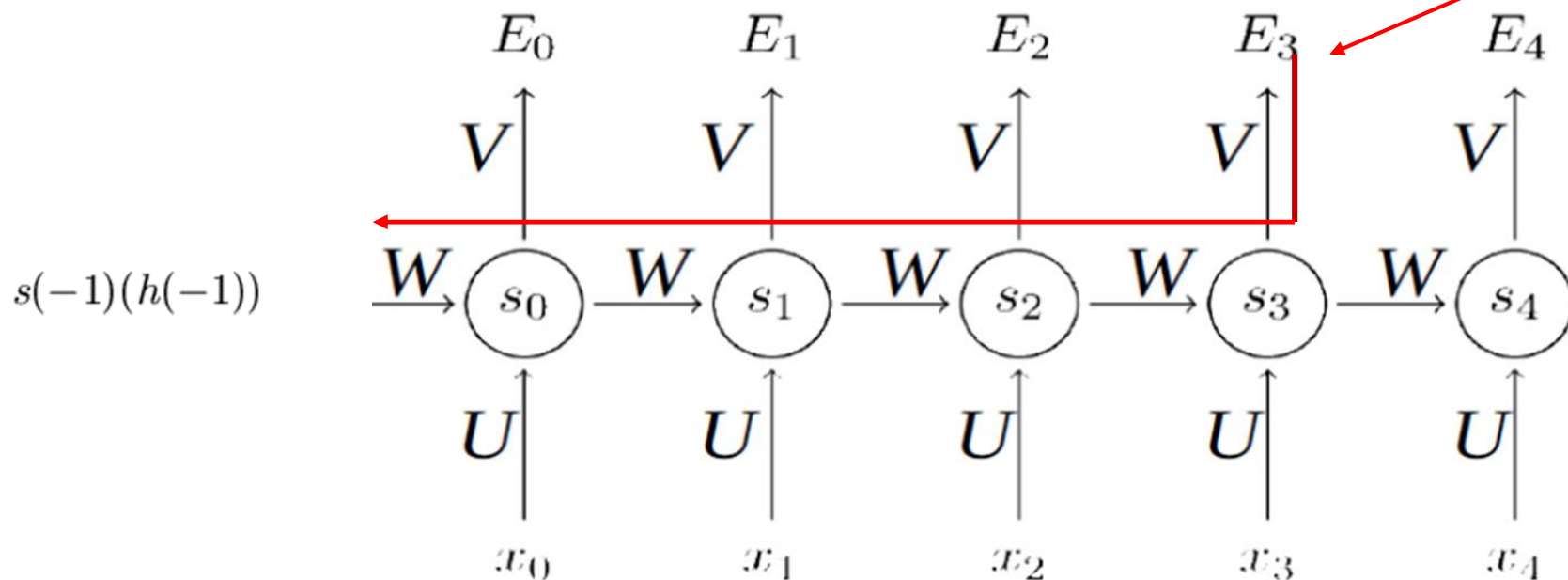
$$\begin{aligned} \frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}} \end{aligned}$$



$$\frac{\partial E}{\partial W_{HH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \cancel{\frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}} \end{aligned}$$

0 if $h_{-1} = 0$



$$\begin{aligned}
\frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\
&= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}
\end{aligned}$$

$$h_3 = f_{OH}(W_{IH}x_3 + W_{HH}h_2) = f_{OH}(v_{H3})$$

$$h_2 = f_{OH}(W_{IH}x_2 + W_{HH}h_1) = f_{OH}(v_{H2})$$

$$h_1 = f_{OH}(W_{IH}x_1 + W_{HH}h_0) = f_{OH}(v_{H1})$$

$$\begin{aligned}\frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}\end{aligned}$$



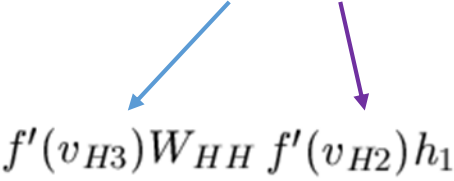
$$f'(v_{H3})h_2$$

$$h_3 = f_{OH}(W_{IH}x_3 + W_{HH}h_2) = f_{OH}(v_{H3})$$

$$h_2 = f_{OH}(W_{IH}x_2 + W_{HH}h_1) = f_{OH}(v_{H2})$$

$$h_1 = f_{OH}(W_{IH}x_1 + W_{HH}h_0) = f_{OH}(v_{H1})$$

$$\begin{aligned}
\frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\
&= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}
\end{aligned}$$



$f'(v_{H3})W_{HH}$ $f'(v_{H2})h_1$

$$h_3 = f_{OH}(W_{IH}x_3 + W_{HH}h_2) = f_{OH}(v_{H3})$$

$$h_2 = f_{OH}(W_{IH}x_2 + W_{HH}h_1) = f_{OH}(v_{H2})$$

$$h_1 = f_{OH}(W_{IH}x_1 + W_{HH}h_0) = f_{OH}(v_{H1})$$

$$\begin{aligned}
\frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\
&= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}
\end{aligned}$$



$f'(v_{H3})W_{HH}$ $f'(v_{H2})W_{HH}$ $f'(v_{H1})h_0$

$$h_3 = f_{OH}(W_{IH}x_3 + W_{HH}h_2) = f_{OH}(v_{H3})$$

$$h_2 = f_{OH}(W_{IH}x_2 + W_{HH}h_1) = f_{OH}(v_{H2})$$

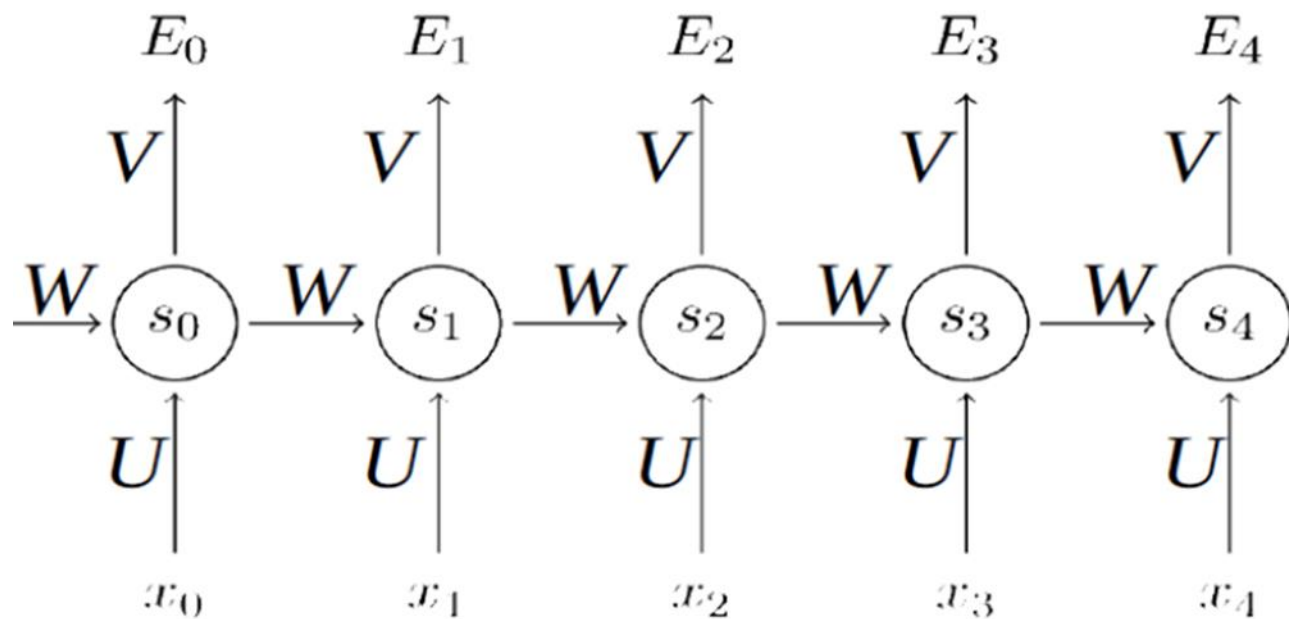
$$h_1 = f_{OH}(W_{IH}x_1 + W_{HH}h_0) = f_{OH}(v_{H1})$$

$$\frac{\partial E}{\partial W_{IH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{IH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{IH}} \end{aligned}$$

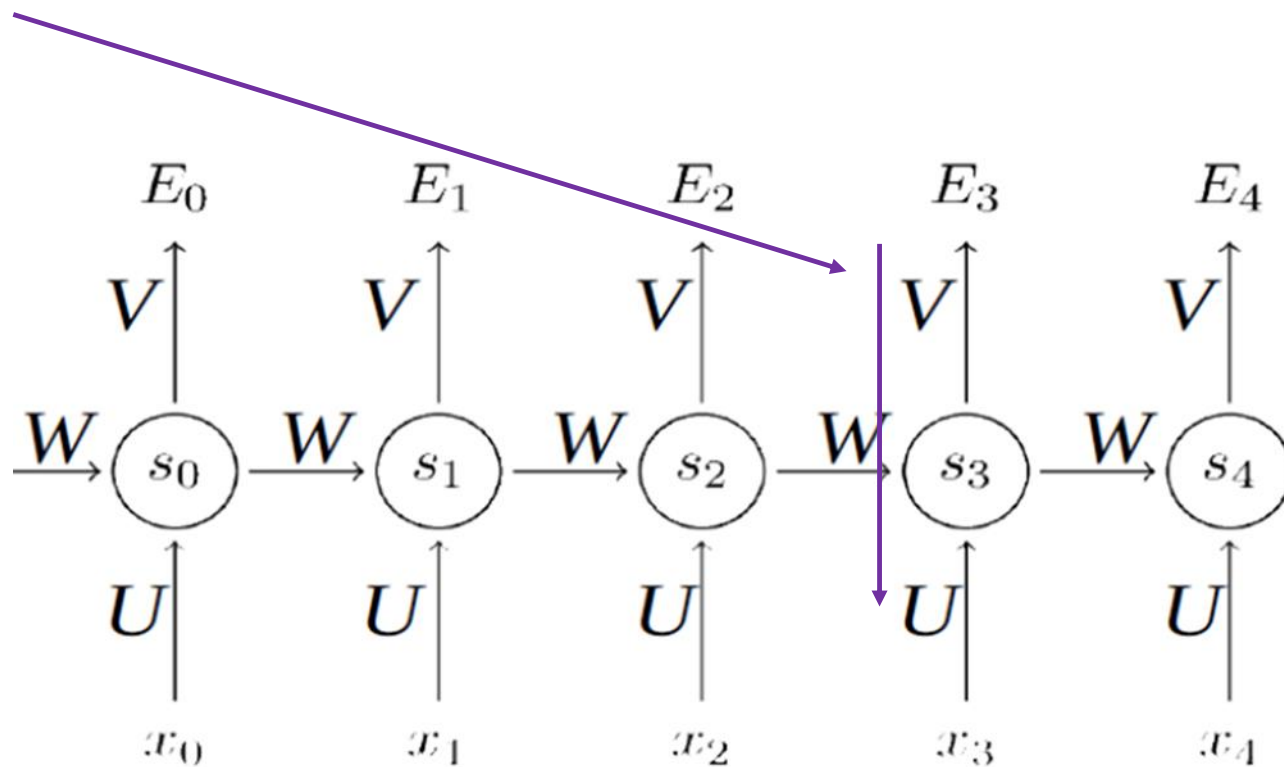
$$\frac{\partial E}{\partial W_{IH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{IH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{IH}} \end{aligned}$$



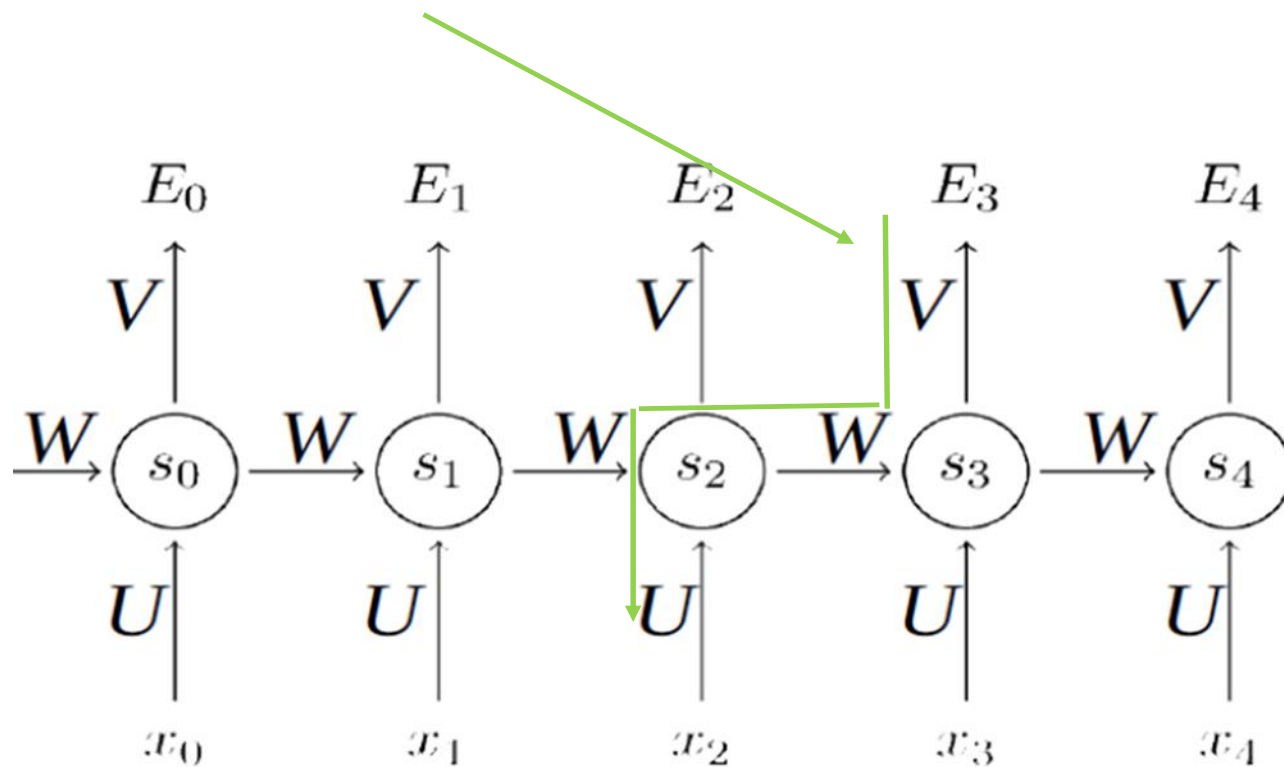
$$\frac{\partial E}{\partial W_{IH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{IH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{IH}} \end{aligned}$$



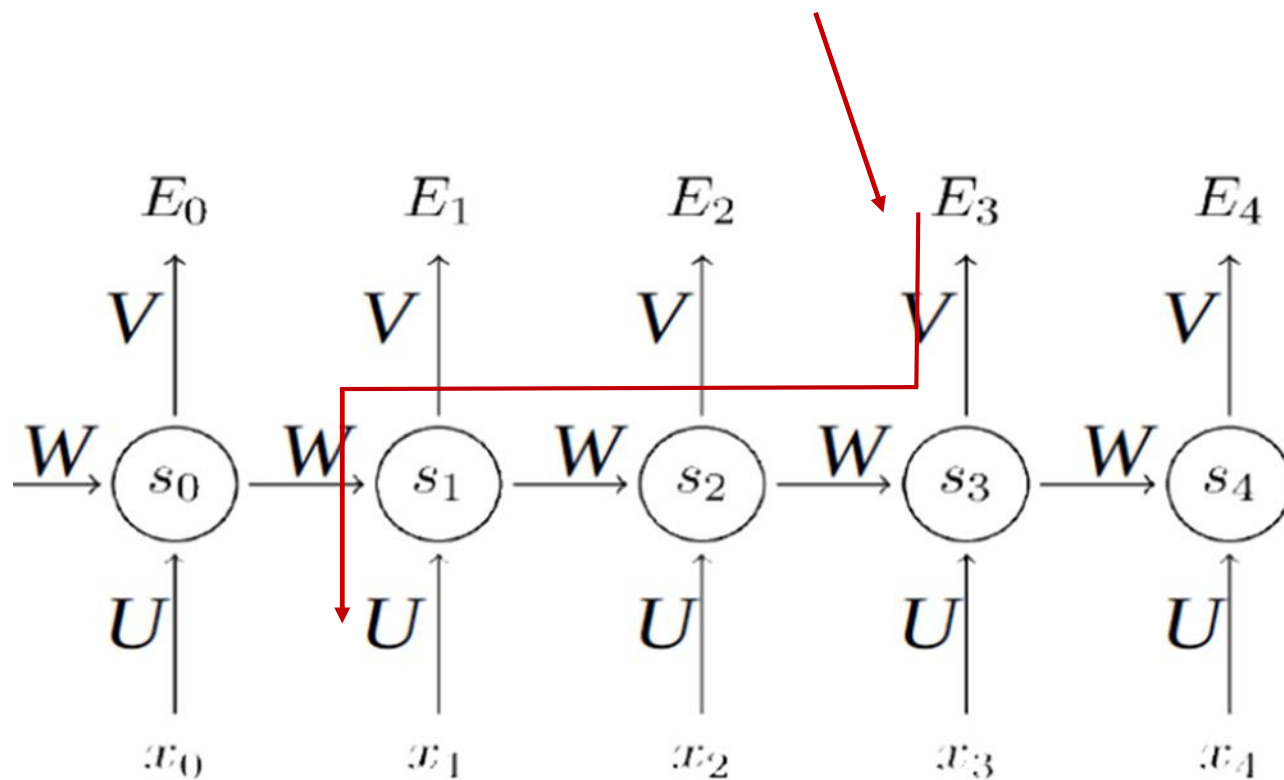
$$\frac{\partial E}{\partial W_{IH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{IH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{IH}} \end{aligned}$$



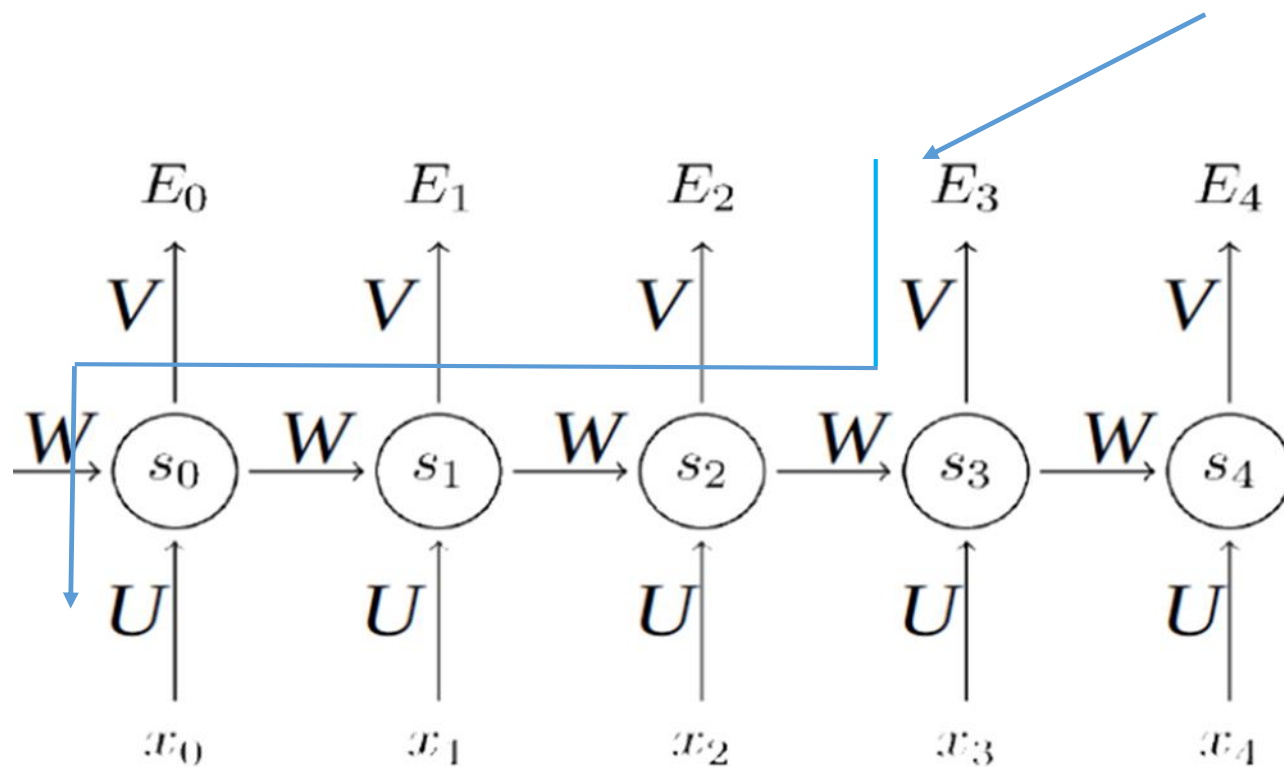
$$\frac{\partial E}{\partial W_{IH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{IH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{IH}} \end{aligned}$$



$$\frac{\partial E}{\partial W_{IH}} = \sum_{i=0}^N \sum_{k=0}^i \frac{\partial e_i}{\partial y_i} \frac{\partial y_i}{\partial h_i} \frac{\partial h_i}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}}$$

$$\begin{aligned} \frac{\partial e_3}{\partial W_{IH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{IH}} \end{aligned}$$



$$\begin{aligned}
\frac{\partial e_3}{\partial W_{IH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{IH}} \\
&= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{IH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{IH}}
\end{aligned}$$

$$h_3 = f_{OH}(W_{IH}x_3 + W_{HH}h_2) = f_{OH}(v_{H3})$$

$$h_2 = f_{OH}(W_{IH}x_2 + W_{HH}h_1) = f_{OH}(v_{H2})$$

$$h_1 = f_{OH}(W_{IH}x_1 + W_{HH}h_0) = f_{OH}(v_{H1})$$

$$h_0 = f_{OH}(W_{IH}x_0 + W_{HH}h_{-1}) = f_{OH}(v_{H0})$$

The Vanishing Gradient Problem

$$\frac{\partial E_3}{\partial \mathbf{W}} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial \mathbf{W}}$$

$$\frac{\partial E_3}{\partial \mathbf{W}} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \left(\prod_{j=k+1}^3 \frac{\partial s_j}{\partial s_{j-1}} \right) \frac{\partial s_k}{\partial \mathbf{W}}$$

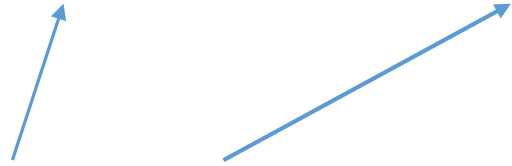
- Derivative of a vector w.r.t a vector is a matrix called jacobian
- 2-norm of the above Jacobian matrix has an upper bound of 1
- **tanh** maps all values into a range between -1 and 1, and the **derivative is bounded by 1**
- With multiple matrix multiplications, gradient values **shrink exponentially**
- Gradient contributions from “far away” steps become zero
- Depending on activation functions and network parameters, gradients could **explode** instead of vanishing

$$\begin{aligned}\frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}\end{aligned}$$

$$f'(v_{H3})h_2 \qquad f'(v_{H3})W_{HH} \, f'(v_{H2})h_1 \qquad f'(v_{H3})W_{HH} \, f'(v_{H2})W_{HH} \, f'(v_{H1})h_0$$

$$\begin{aligned}\frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}\end{aligned}$$

$$f'(v_{H3})h_2 \quad f'(v_{H3})W_{HH} f'(v_{H2})h_1 \quad f'(v_{H3})W_{HH} f'(v_{H2})W_{HH} f'(v_{H1})h_0$$



Gradient multiples may get smaller

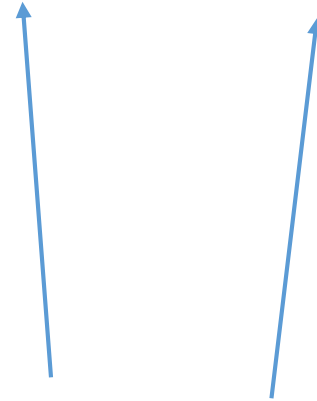
Matrix multiplications may get larger

$$\begin{aligned}\frac{\partial e_3}{\partial W_{HH}} &= \sum_{k=0}^3 \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{HH}} \\ &= \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{HH}} + \frac{\partial e_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_0} \frac{\partial h_0}{\partial W_{HH}}\end{aligned}$$

$$f'(v_{H3})h_2 \quad f'(v_{H3})W_{HH} f'(v_{H2})h_1 \quad f'(v_{H3})W_{HH} f'(v_{H2})W_{HH} f'(v_{H1})h_0$$

Gradient multiples may get smaller

Matrix multiplications may get larger



RNN Tricks

(Pascanu et al., 2013; Bengio et al., 2013; Gal and Ghahramani, 2016; Morishita et al., 2017)

- Mini-batch creation strategies (efficient computations)
- Clipping gradients (avoid exploding gradients)
- Leaky integration (propagate long-term dependencies)
- Momentum (cheap 2nd order)
- Dropout (avoid overfitting)
- Initialization (start in right ballpark avoids exploding/vanishing)
- Sparse Gradients (symmetry breaking)
- Gradient propagation regularizer (avoid vanishing gradient)
- Gated self-loops (LSTM & GRU, reduces vanishing gradient)

Truncated Time steps

The idea is to stop computing the sum of gradients after time step τ , this leads to an approximation of the true gradients and gives quite good results in practice. This version of BPTT is called **Truncated Backpropagation Through Time** (referred as **TBPTT**) [2]. As a consequence to this method, the model starts focusing on short term influence rather than long-term ones and with that the model becomes biased. To solve this problem, the researchers suggested to use subsequences with variable length instead of using fixed length subsequences. This method is explained in the following paragraph.

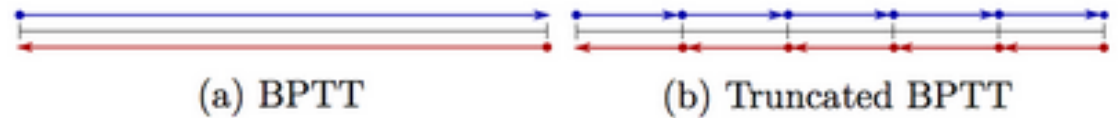


Figure 1: Graphical representation of BPTT, truncated BPTT

Blue arrows represent forward propagations, red arrows backpropagations.

Dots represent either internal state resetting or gradient resetting.

https://iq.opengenus.org/back-propagation-through-time/#google_vignette

3.2. Scaling down the gradients

As suggested in section 2.3, one simple mechanism to deal with a sudden increase in the norm of the gradients is to rescale them whenever they go over a threshold (see algorithm 1).

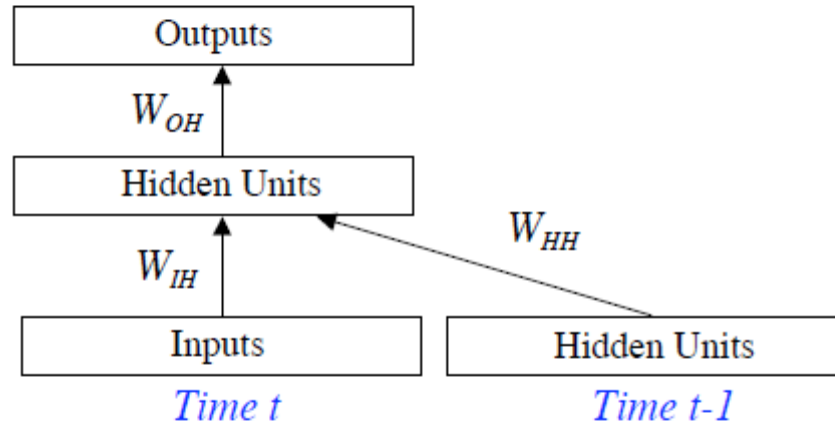
Algorithm 1 Pseudo-code for norm clipping the gradients whenever they explode

```
 $\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$   
if  $\|\hat{\mathbf{g}}\| \geq threshold$  then  
     $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$   
end if
```

<https://arxiv.org/pdf/1211.5063.pdf>

Simple Recurrent Networks

Truncating the unfolded network to just one time step reduces it to a so-called *Simple Recurrent Network* (which is also commonly known as an *Elman network*):



In this case, each set of weights now only appears only once, so it is possible to apply the gradient descent approach using the standard backpropagation rather than full BPTT.

In practice, this approximation proves to be too great for many practical applications.



The Unreasonable Effectiveness of Recurrent Neural Networks

May 21, 2015

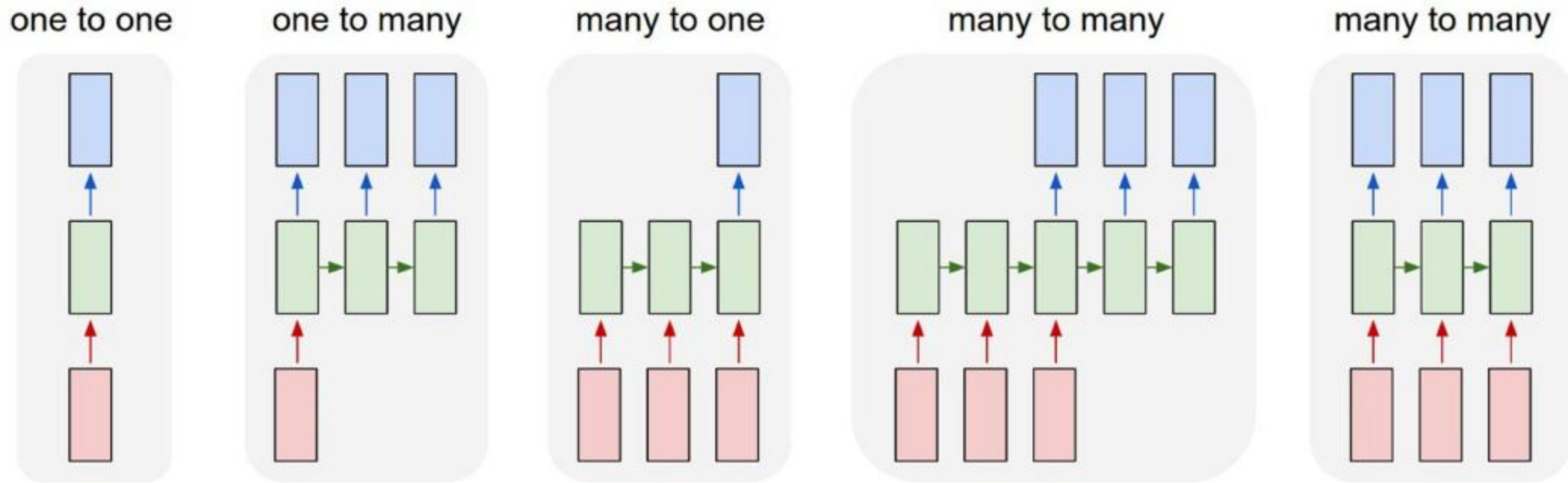
There's something magical about Recurrent Neural Networks (RNNs). I still remember when I trained my first recurrent network for [Image Captioning](#). Within a few dozen minutes of training my first baby model (with rather arbitrarily-chosen hyperparameters) started to generate very nice looking descriptions of images that were on the edge of making sense. Sometimes the ratio of how simple your model is to the quality of the results you get out of it blows past your expectations, and this was one of those times. What made this result so shocking at the time was that the common wisdom was that RNNs were supposed to be difficult to train (with more experience I've in fact reached the opposite conclusion). Fast forward about a year: I'm training RNNs all the time and I've witnessed their power and robustness many times, and yet their magical outputs still find ways of amusing me. This post is about sharing some of that magic with you.

We'll train RNNs to generate text character by character and ponder the question "how is that even possible?"

By the way, together with this post I am also releasing [code on Github](#) that allows you to train character-level language models based on multi-layer LSTMs. You give it a large chunk of text and it will learn to generate text like it one character at a time. You can also use it to reproduce my experiments below. But we're getting ahead of ourselves; What are RNNs anyway?

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

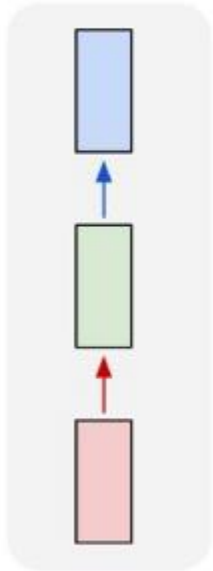
Recurrent Networks offer a lot of flexibility:



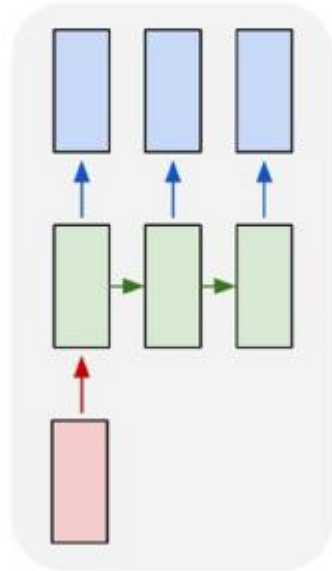
Vanilla Neural Networks

Recurrent Networks offer a lot of flexibility:

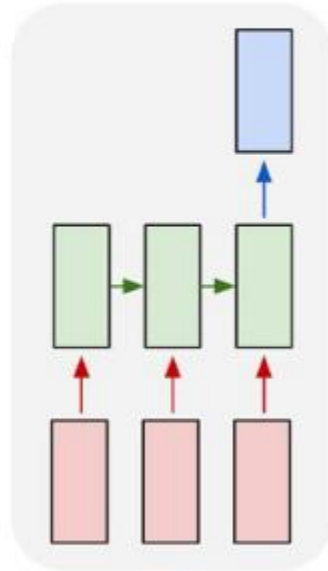
one to one



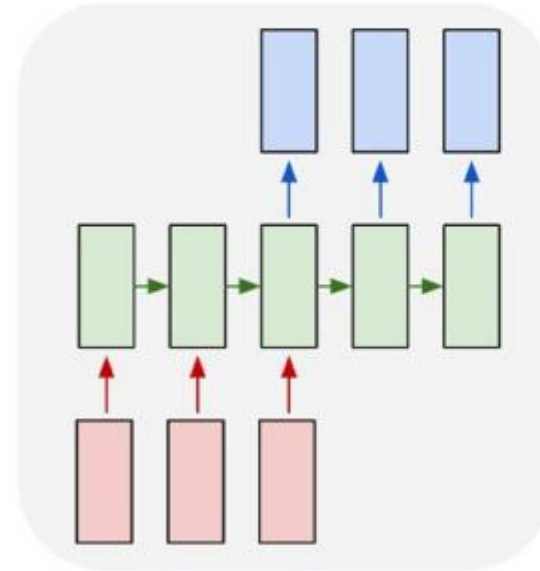
one to many



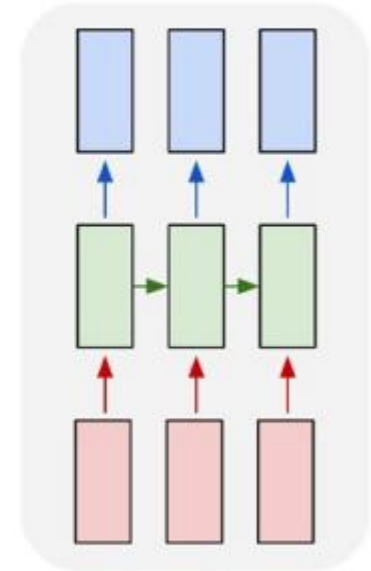
many to one



many to many



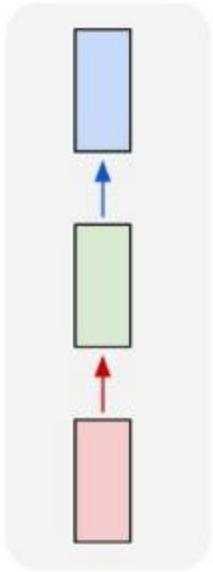
many to many



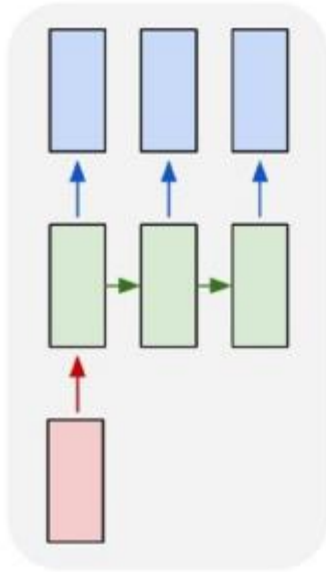
↖ e.g. **Image Captioning**
image -> sequence of words

Recurrent Networks offer a lot of flexibility:

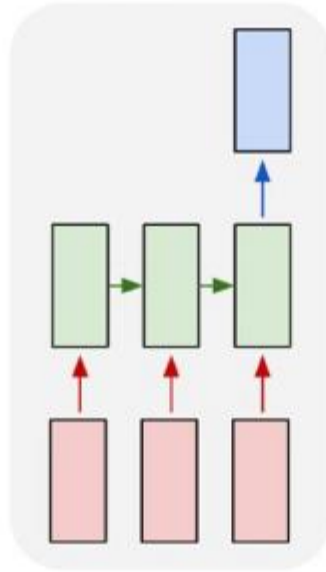
one to one



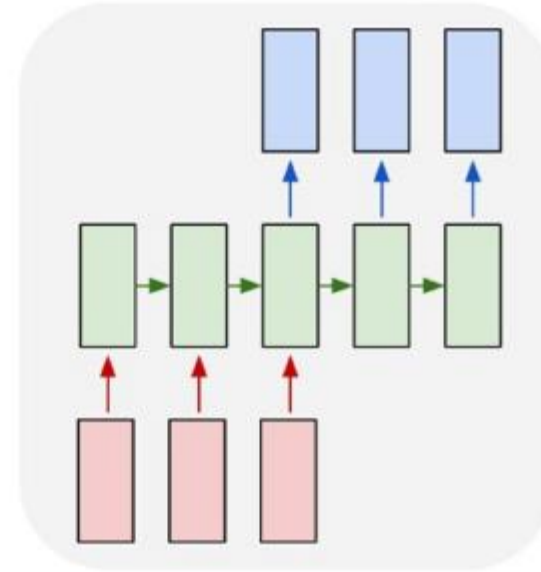
one to many



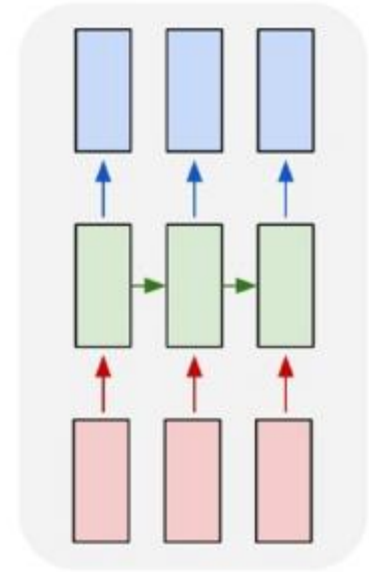
many to one



many to many



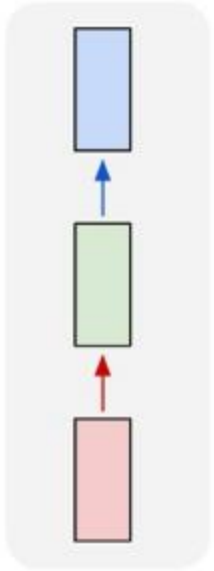
many to many



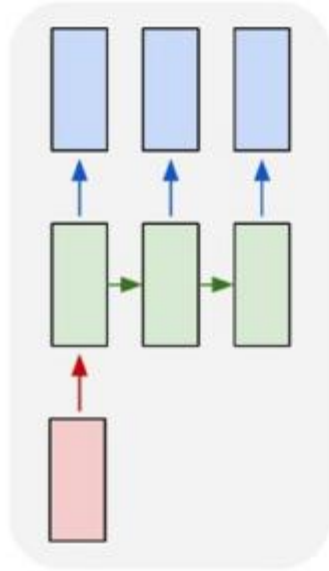
↖ e.g. **Sentiment Classification**
sequence of words -> sentiment

Recurrent Networks offer a lot of flexibility:

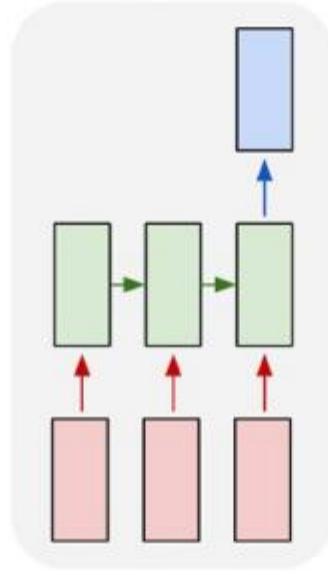
one to one



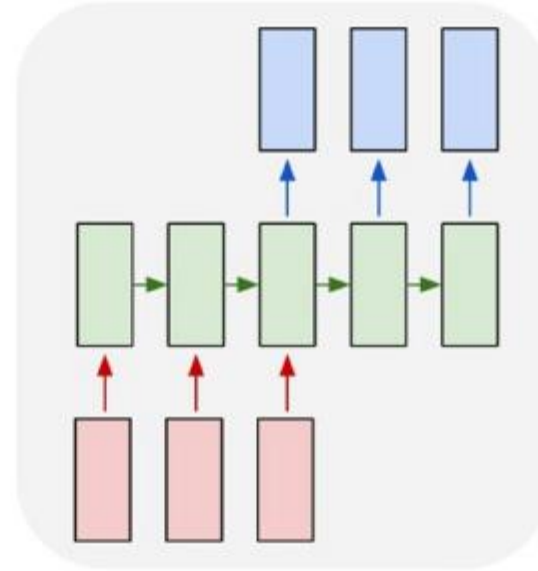
one to many



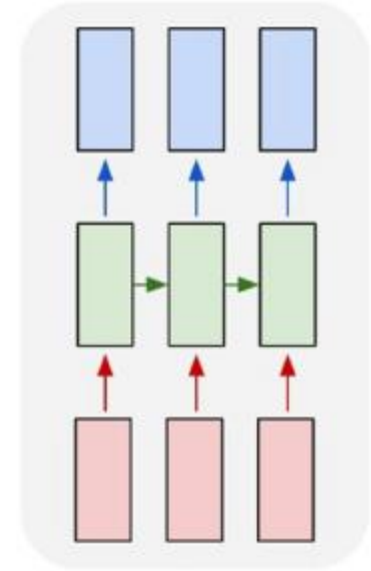
many to one



many to many



many to many

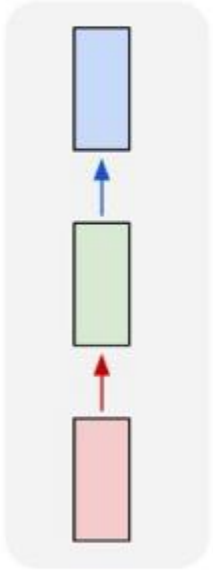


e.g. **Machine Translation**

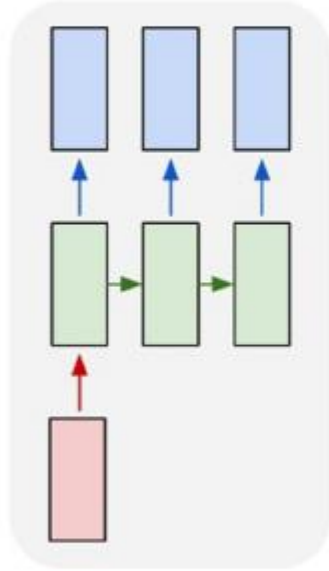
seq of words -> seq of words

Recurrent Networks offer a lot of flexibility:

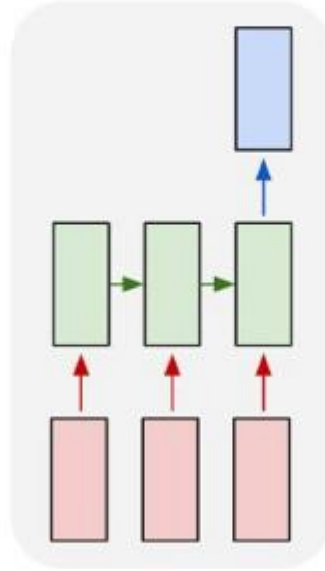
one to one



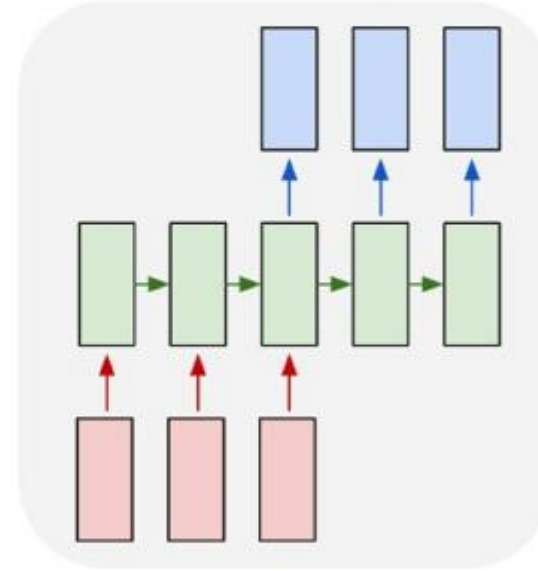
one to many



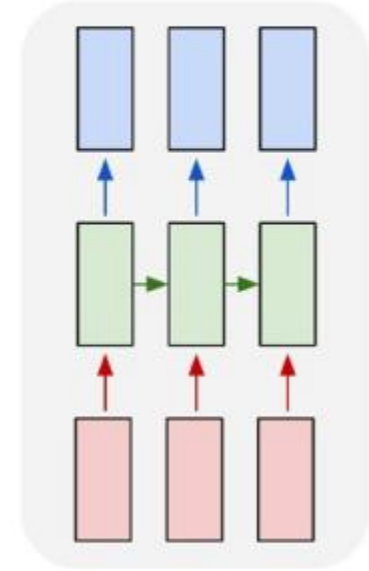
many to one



many to many



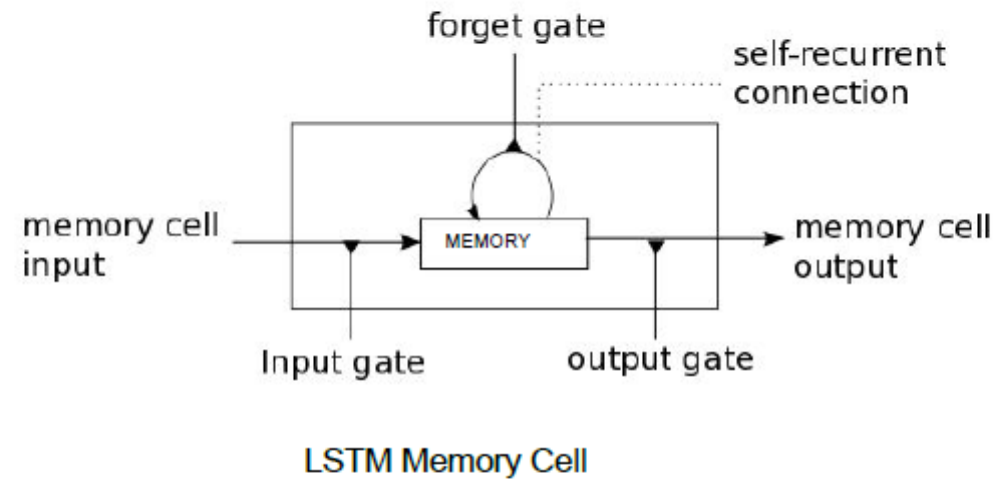
many to many



e.g. Video classification on frame level

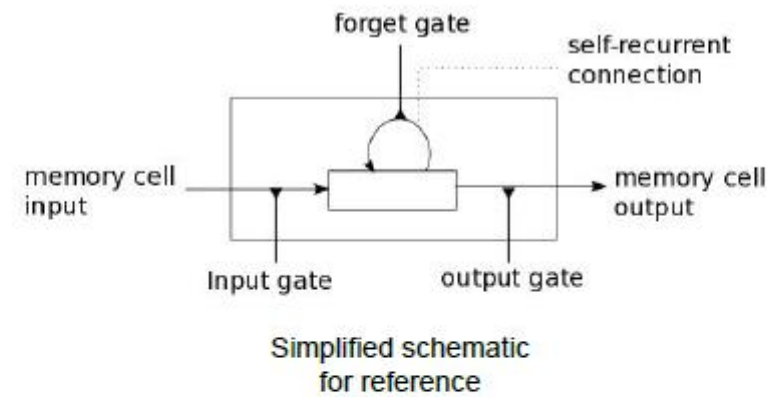
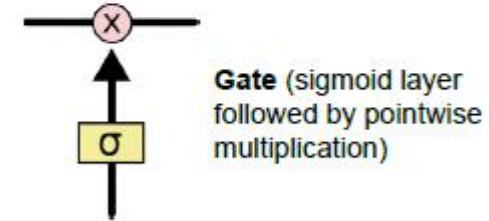
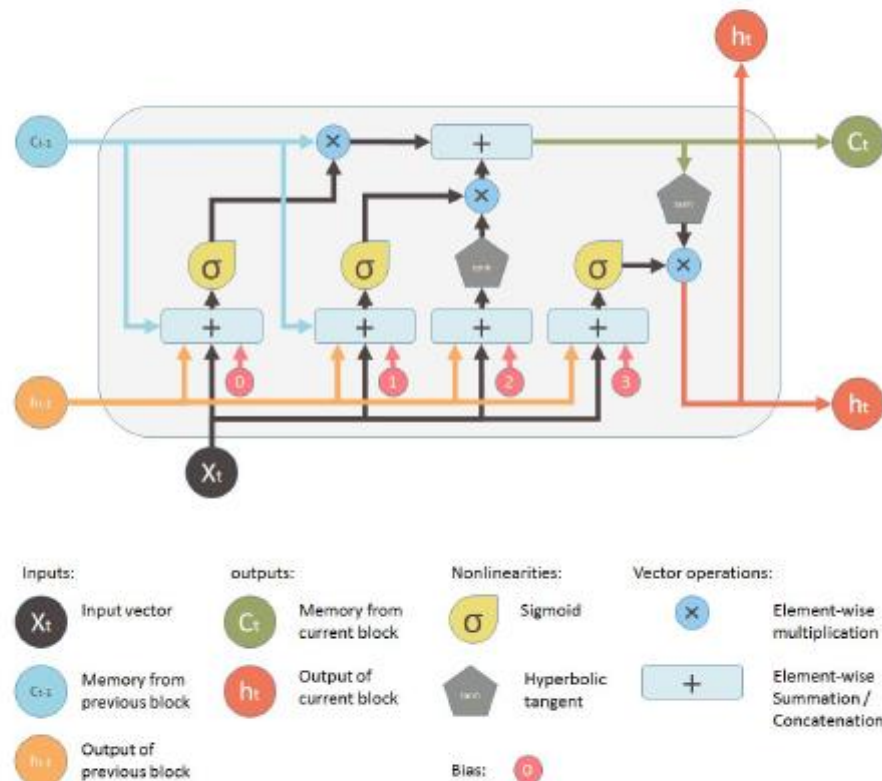


Long Short-Term Memory networks (LSTMs)



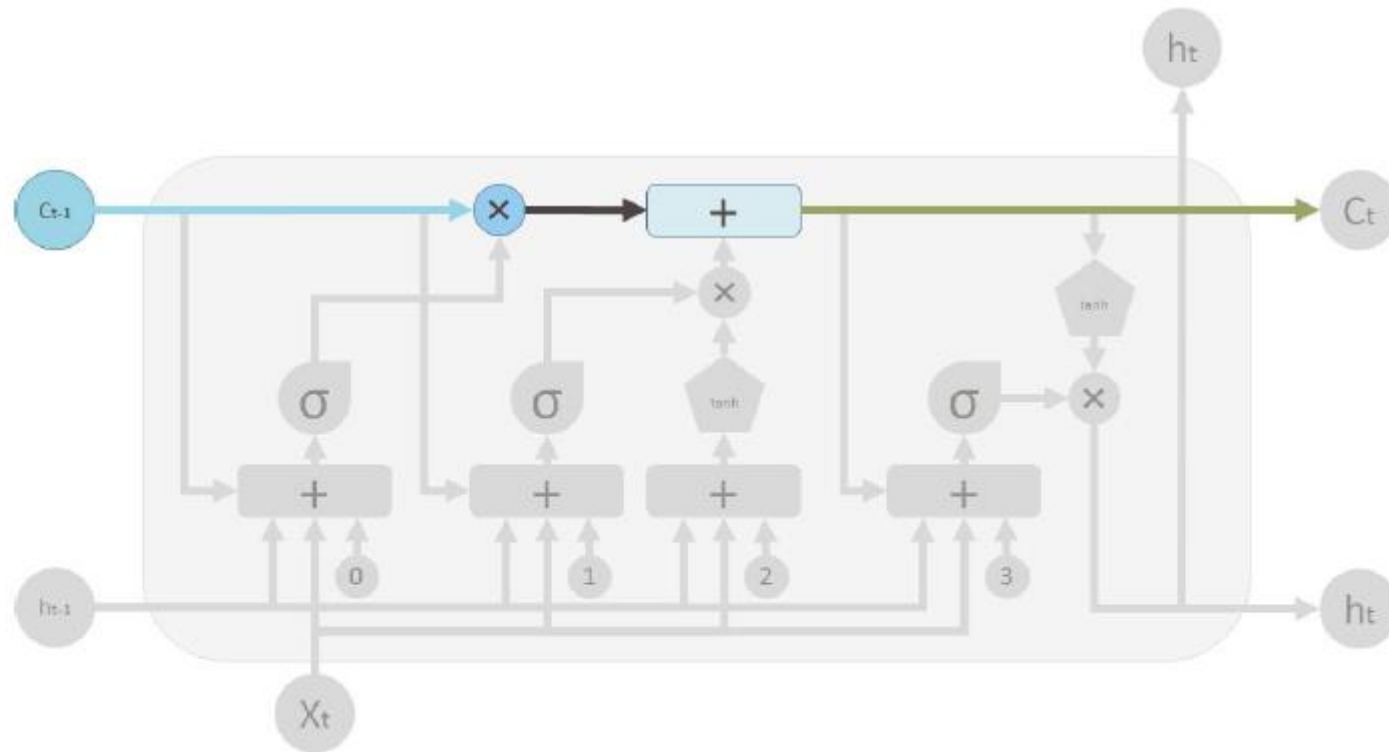
Central Idea: A **memory cell** (interchangeably **block**) which can maintain its state over time, consisting of an explicit memory (aka the **cell state vector**) and **gating units** which regulate the information flow into and out of the memory.

Long Short-Term Memory Cell

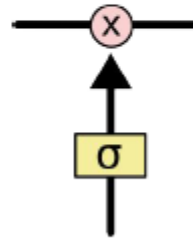


Cell State Vector

- Represents the memory of the LSTM
- Undergoes changes via forgetting of old memory (forget gate) and addition of new memory (input gate)



Gates

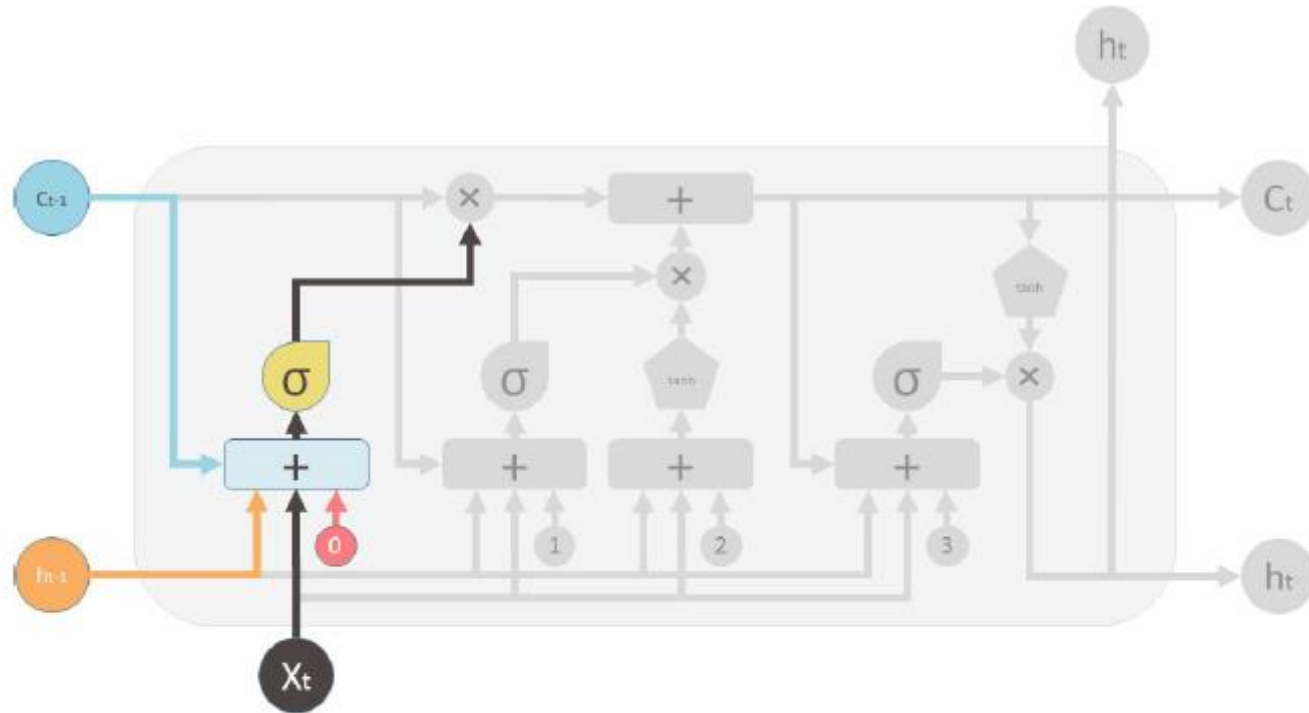


- Gate: sigmoid neural net layer followed by pointwise multiplication operator
- Gates control the flow of information to/from the memory
- Gates are controlled by a concatenation of the output from the previous time step and the current input and optionally the cell state vector.

Forget Gate

- Controls what information to throw away from memory

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

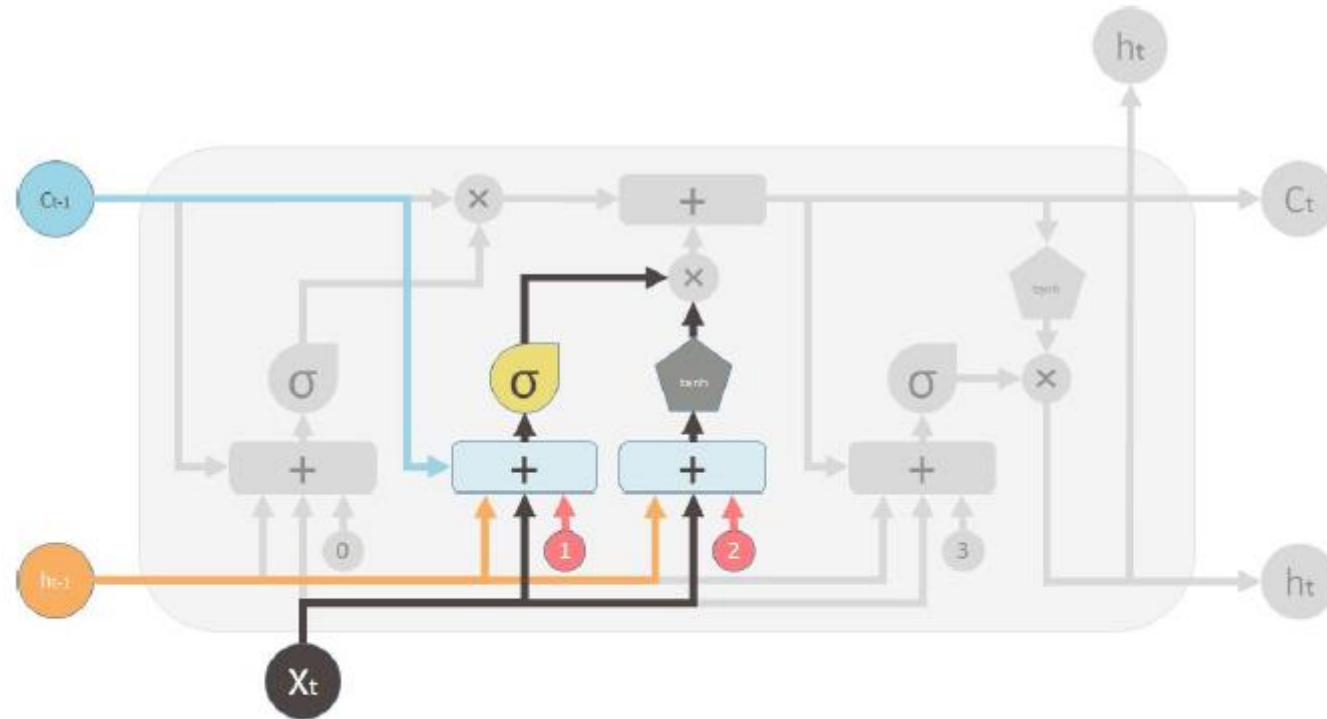


Input Gate

- Controls what new information is added to cell state from current input

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

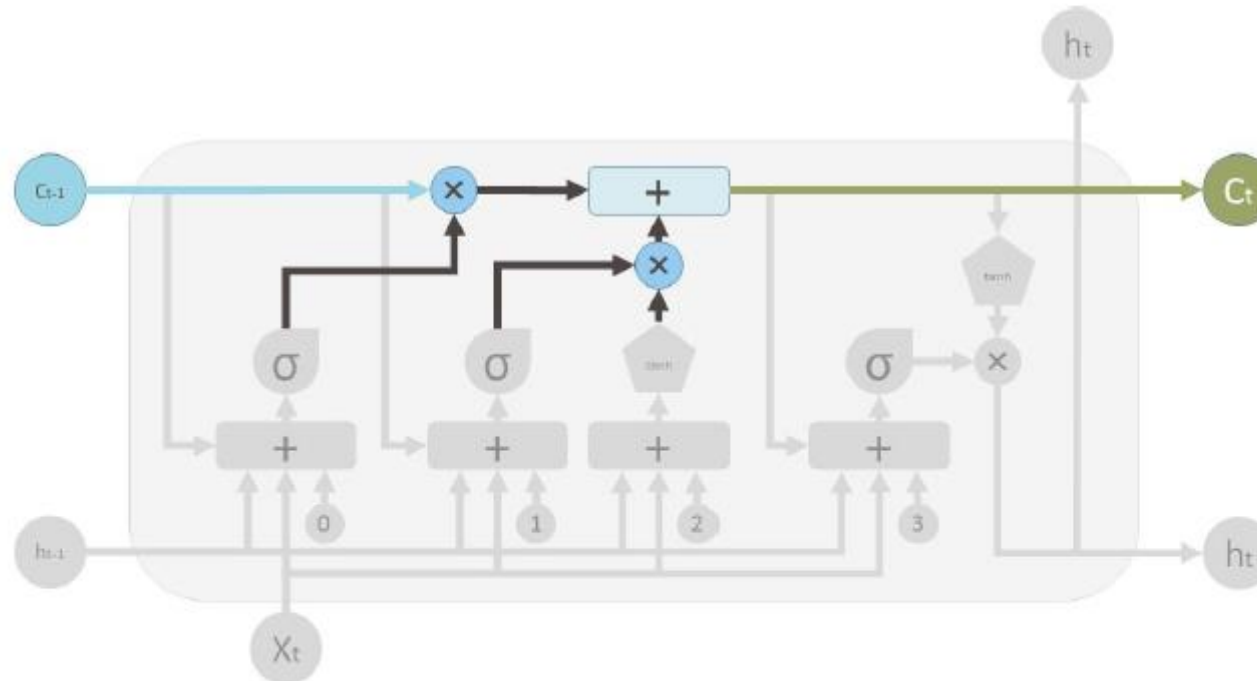
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



Memory Update

- The cell state vector aggregates the two components (old memory via the forget gate and new memory via the input gate)

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

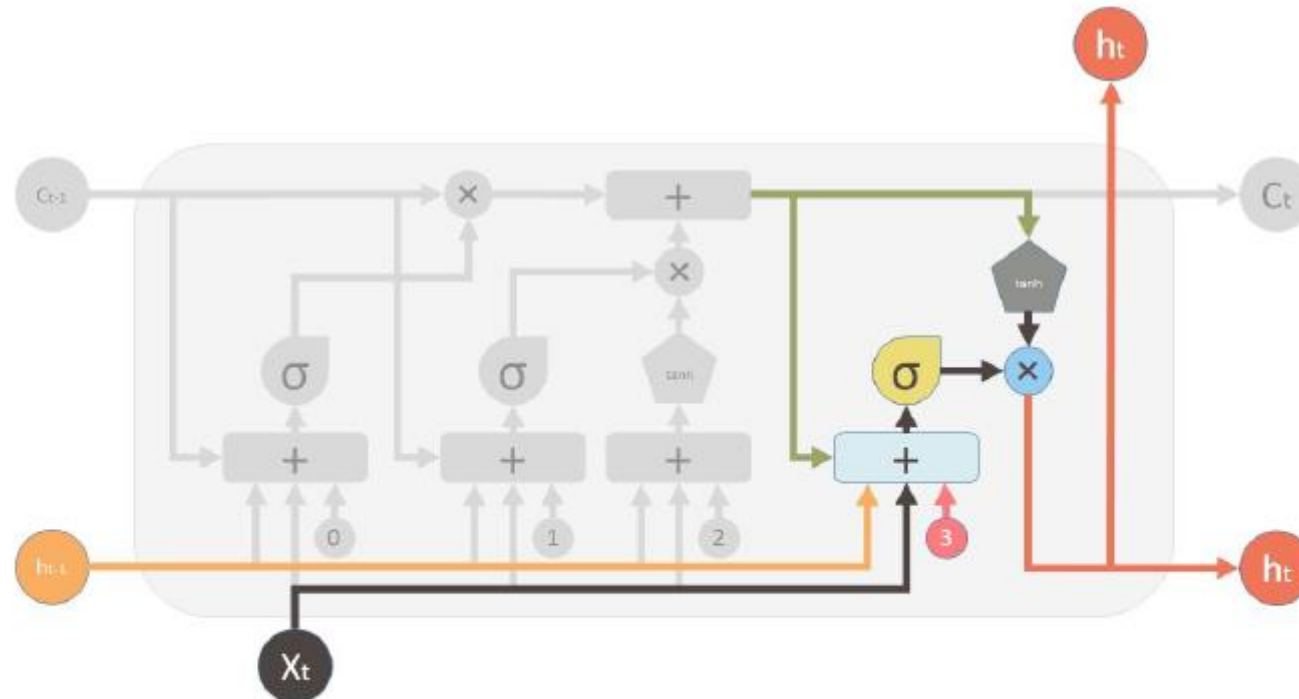


Output Gate

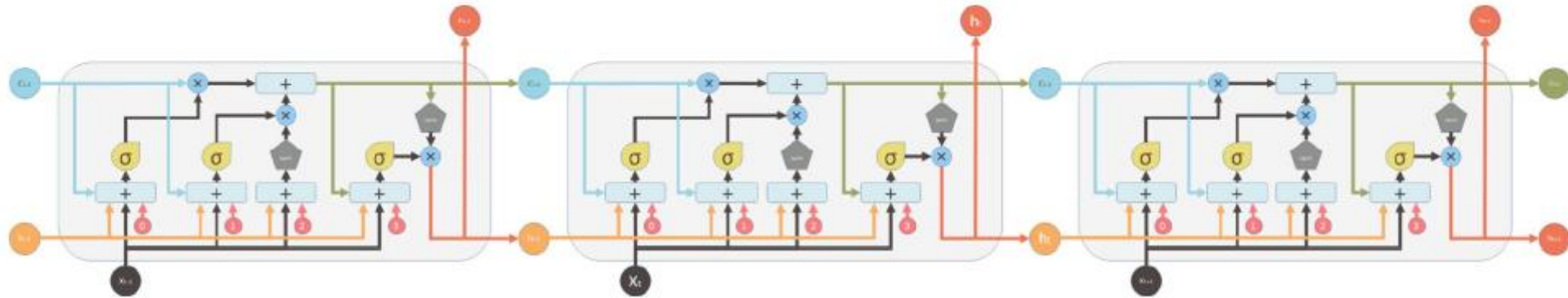
- Conditionally decides what to output from the memory

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



LSTM Layer



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

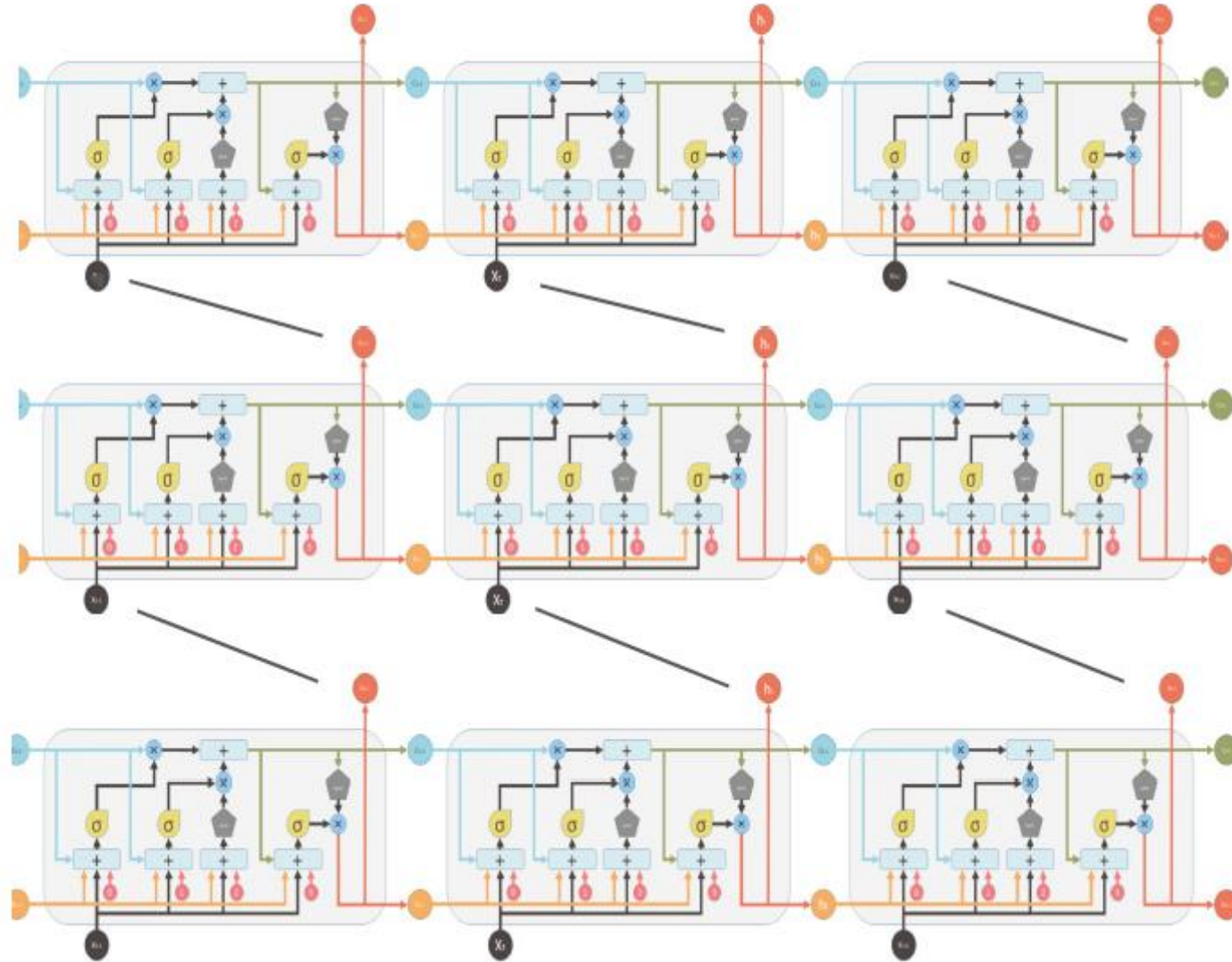
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

LSTM Network



Hopfield Networks

The *Hopfield Network* or *Hopfield Model* is one good way to implement an associative memory. It is simply a fully connected recurrent network of N McCulloch-Pitts neurons. Activations are normally ± 1 , rather than 0 and 1, so the neuron activation equation is:

$$x_i = \text{sgn}\left(\sum_j w_{ij}x_j - \theta_i\right) \quad \text{where} \quad \text{sgn}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Hopfield Networks

The *Hopfield Network* or *Hopfield Model* is one good way to implement an associative memory. It is simply a fully connected recurrent network of N McCulloch-Pitts neurons. Activations are normally ± 1 , rather than 0 and 1, so the neuron activation equation is:

$$x_i = \text{sgn}\left(\sum_j w_{ij}x_j - \theta_i\right) \quad \text{where} \quad \text{sgn}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

The activations keep changing till they have all settled down to some stable pattern. Those activations can be updated either *synchronously* or *asynchronously*.

Hopfield Networks

The *Hopfield Network* or *Hopfield Model* is one good way to implement an associative memory. It is simply a fully connected recurrent network of N McCulloch-Pitts neurons. Activations are normally ± 1 , rather than 0 and 1, so the neuron activation equation is:

$$x_i = \text{sgn}\left(\sum_j w_{ij}x_j - \theta_i\right) \quad \text{where} \quad \text{sgn}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

The activations keep changing till they have all settled down to some stable pattern. Those activations can be updated either *synchronously* or *asynchronously*.

It can be shown that the required associative memory can be achieved by simply setting the weights w_{ij} and thresholds θ_j in relation to the target outputs $\mathbf{t}^p$ as follows:

$$w_{ij} = \frac{1}{N} \sum_{p=1}^P t_i^p t_j^p, \quad \theta_i = 0$$